矽璘操作系统原理

目录

第 1 章 前言.....3
第 2 章 XiUOS 基本介绍.....3
2.1 背景.....3
2.2 定位.....3
2.3 特色.....4
2.4 部署.....5
2.4.1 海尔热水器厂工艺数字化.....5
2.4.2 大胜达智能环境感知系统.....6
第 5 章 架构.....7
第 3 章 XiZi 内核原理.....8
3.1 线程管理.....9
3.1.1 线程数据结构.....9
3.1.2 线程状态切换.....12
3.1.3 线程通信对象.....13
3.1.4 线程隔离机制.....18
3.1.5 线程管理接口.....21
3.2 内存管理.....23
3.2.1 内存数据结构.....23
3.2.2 内存管理机制.....26
3.2.3 内存管理接口.....30
3.3 中断管理.....32
3.3.1 中断数据结构.....32
3.3.2 中断管理机制.....34
3.3.3 中断管理接口.....40
3.4. 时钟管理.....41
3.4.1 时钟数据结构.....41
3.4.2 时钟管理机制.....42
3.4.3 时钟管理接口.....43
3.5 文件系统.....44
3.5.1 文件系统数据结构.....45
3.5.2 虚拟文件系统.....47
3.5.3 文件系统接口.....49

3.6 驱动管理.....50
3.6.1 驱动模型数据结构.....51
3.6.2 驱动模型.....53
3.6.3 驱动模型接口.....54

第 1 章 前言

第 2 章 XiUOS 基本介绍

2.1 背景

2022 年 1 月，梅宏院士等在中国科学院院刊上发表了《泛在操作系统：面向人机物融合泛在计算的新蓝海》署名文章，总结了操作系统发展的“20 年周期律”，指出泛在计算时代操作系统发展的新蓝海。这是继 2018 年首次在 IEEE Computer 提出泛在操作系统的概念之后，对人机物融合的泛在操作系统发展指明了新的方向。

矽璘（XiUOS）是面向工业物联网的泛在操作系统，前期基础主要来自北京大学在网构操作系统以及上海交通大学在软件定义内存计算系统等方面的长期技术积累。2019 年 11 月，矽璘的研制在浙江省北大信息技术高等研究院启动，2021 年 5 月，开源发布 XiUOS-1.0，2021 年 12 月，开源发布 XiUOS-2.0。矽璘致力于解决工业物联场景中面临的全面感知、泛在互联、实时识别、精准控制等问题。

2.2 定位

XiUOS 是一种工业物联网操作系统，目标是通过工业物联网的部署和应用，促进工业领域人、机、物的深度互联，促进工厂的数字化转型升级，使能智能化工业生产新体系。要实现该目标，工业物联网操作系统需要解决“感、联、知、控”四方面的问题：

- 感：复杂多样的工业生产实体智能地识别、感知和采集生产相关数据；
- 联：工业数据在互联互通的网络上进行传输和汇聚；
- 知：对这些网络化的工业大数据形成快速处理和实效分析；
- 控：将数据分析所得到的信息反馈到工业生产中，对生产活动进行改善。

其中，“感”和“联”主要在物联网节点级设备、传感器等设备上实现，“知”和“控”的核心主要在边缘计算设备和数据中心设备上实现。

面向物联网节点级设备的矽璘工业物联操作系统，主要定位于“感”、“联”和“控”的层次，支持接入工业场景下各种传感器，将感知获得的数据传递给边缘设备和数据中心设备，同时具有对 PLC、工控机等工控设备的控制能力，能对“知”提供支持，如图 2-3-1 所示。

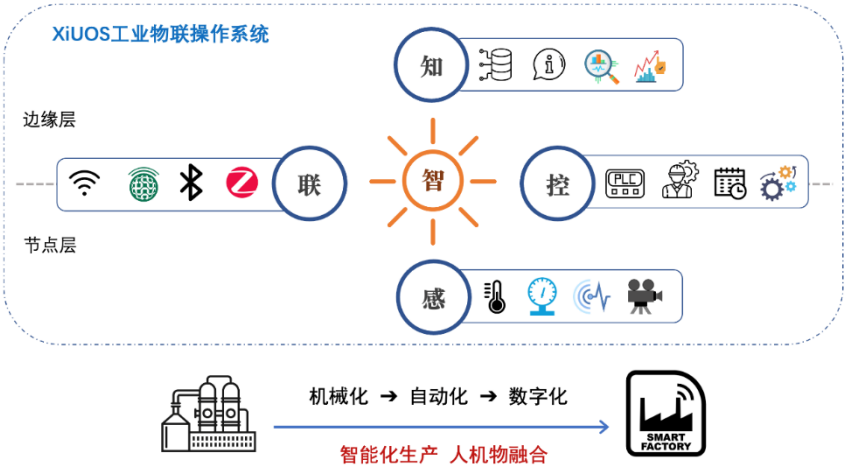


图 2-3-1 “感联知控”应用框架

2.3 特色

目前市面上已有非常多成功的物联网操作系统，如国外的 FreeRTOS、Zephyr、ROS、µC/OS，国内的 RT-Thread、LiteOS、AliOS Things 等，这些系统多是从实时嵌入式系统（RTOS: Real time OS）发展而来，通常具有非常小巧的实时内核，多运行在资源受限的微控制单元（MCU：Microcontroller Unit）或单片机上，对低功耗有较高要求，常见的处理器为 ARM Cortex-M、RISC-V 系列。

矽璘工业物联操作系统 XiUOS 和现有大多数的物联网 OS 关注点并不相同，具有自己的特色：

- XiUOS 不仅着眼于内核的基础功能，如简单实时的调度，静态内存的预分配与动态内存的碎片合并相结合的机制，任务隔离巧妙耦合硬件体系结构特征的算法，而且更关注基础功能之上的业务驱动的高级功能，如以感知数据本身为中心的传感器驱动方法，以方便用户设备联网的无线网络自动接入功能，在节点端启动图像与声音识别的智能化功能；
- XiUOS 不仅注重于 OS 本身在多领域的通用性，而且更寻求其在工业物联领域的独特性，将复杂的工业控制流程解耦为多个不可再分的“控制元”，以软件定义的方式对各种“控制元”进行再组合或构造，从而在节点端完成对工业设备的实时控制，满足工业企业的对精准控制的刚需；
- XiUOS 不仅关切单个节点设备本身功能和效用的发挥，而且更聚焦于在大规模工业生产环境中组成的网络化集群，强调对大规模部署和管理运维的支持，强调节点设备群构成的物联体系对工业生产的促进作用；
- XiUOS 吸取现有闭源和开源 RTOS 在各自细分领域的特征，更加关注生态的开放性，依托并逐步融入现有的主流开源生态体系，形成其在工业领域的重要角色和作用。

综合以上描述，XiUOS 是软硬件协同的物联网解决方案，提供自研的有特色的硬件系统，支持 ARM 和 RISC-V 两种体系结构的处理器，在同一物理终端上同时支持多种通信方式和多种总线接口，可直接部署于工业生产环境，使得工业物联网用户不必关心底层各类硬件的实施细节，从而更加专注于应用逻辑本身的效能。

简而言之，现有物联网 OS 主要关注的是做优秀的单节点 OS，XiUOS 是在现有单节点 OS 的基础上构建面向多节点的网络化智能物联软件栈，为工业物联网领域应用开发、系统部署、管理运维提供友好方案，因此现有物联网 OS 和 XiUOS 功能是主要是互补的而非竞

争的，如 Linux 之于 Android。

2.4 部署

XiUOS 是面向工业领域多节点设备的网络化 OS，其部署主要包括节点端的多节点集群和边缘端/云端的计算设施，如图 2-4-1 所示。

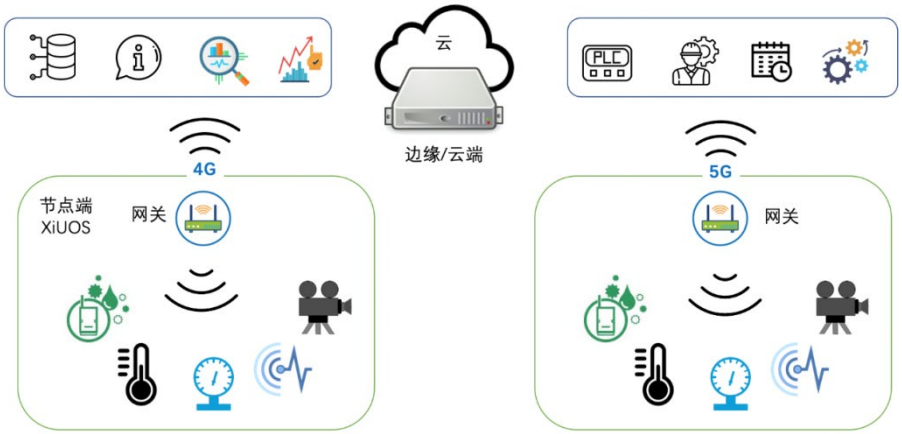


图 2-4-1 XiUOS 部署环境示意图

在节点端，支持多种工厂环境传感设备，既包括基础的环境传感器感知温湿度、烟雾气体、噪音等，也包括智能传感器可对人体、物体、声音进行识别，这些传感器支持多种网络通信互联协议，可以按需自组网络，通过网关设备向外交换数据，实现智能物联框架的“感、联”功能。网关设备对同一个集群中的节点设备进行管理，负责和边缘/云端系统交换数据，并可具备有限的“知、控”功能。

节点端感知的数据经网关传输给边缘/云端系统之后，边缘/云端的工业互联网平台可对汇聚而来的数据进行分析，将有价值的数据在控制中心的大屏、管理人员的移动设备上进行展示，并根据工业应用的业务规则，通知和调度“设备、排程、人员”，对生产过程进行优化，完成“知、控”。

目前，XiUOS 已在山东青岛海尔热水器厂产线和浙江大胜达包装股份有限公司彩印车间等场景部署了示范应用。

2.4.1 海尔热水器厂工艺数字化

海尔热水器厂工艺数字化方案主要负责采集卷圆、直缝焊、扩口、组队、环缝、水管焊接、试压、打砂、搪瓷、预装、氮氢试压、发泡机和打包机等产线工位的 PLC 数据采集，并通过无线蜂窝网络 4G/5G、短距离无线 LoRa 网络和以太网等多种通信方式上传至海尔智家平台数据库，实时监控各工位生产状况和进度，如图 2-4-2 所示。

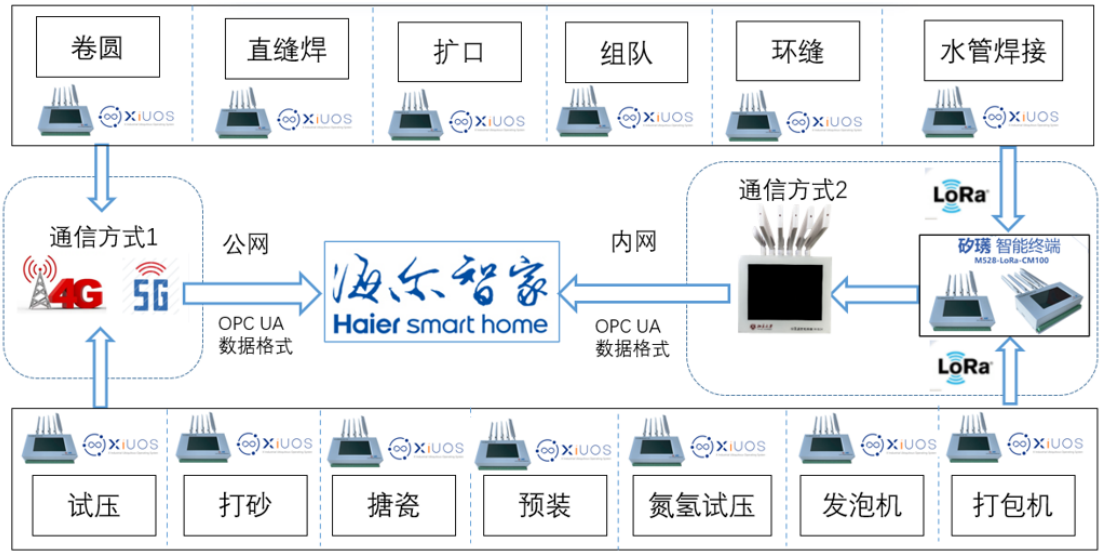


图 2-4-2 海尔热水器厂工艺数字化示意图

其中，矽璘工业物联智能终端运行 XiUOS，支持丰富的外设驱动接口，与产线工位 PLC 网口连接，解析通信协议并组装协议报文，实时采集 PLC 节拍数据，同时支持稳定的无线 LoRa 网络模组，通过私有协议组网上传数据至 LoRa 网关，根据 OPC UA 数据格式要求生成数据报文，存入海尔智家数据库。

2.4.2 大胜达智能环境感知系统

大胜达智能环境感知系统主要采集 PM1.0、PM2.5、PM10、AQS、TVOC、甲醛、乙醇、二氧化碳、噪音、甲烷、温度、湿度、气压、风速、风向等环境因子，此外还包括部分机器电能消耗的监测。该系统主要由 1 个网关、1 个电表监测节点和 6 组环境监测节点组成，每组包含 8 个监测节点，共 50 台设备，覆盖整个彩印车间，监测环境因子，如图 2-4-3 所示。

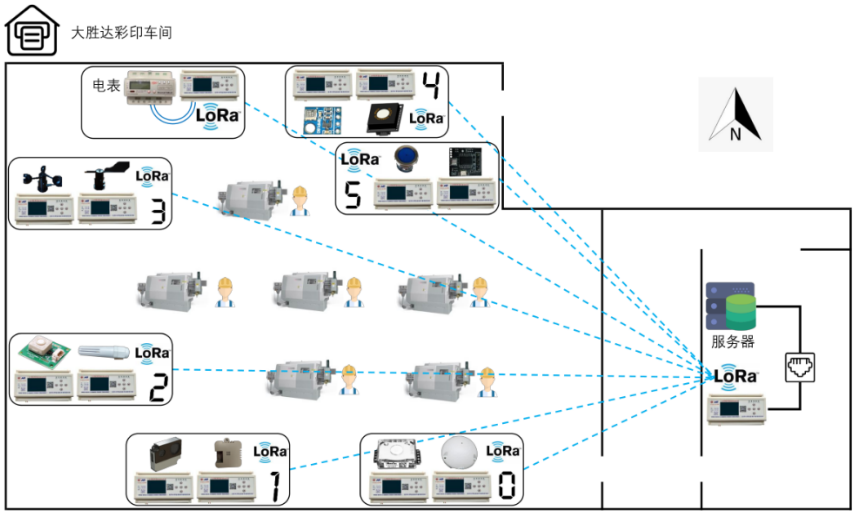


图 2-4-3 大胜达智能环境感知系统示意图

其中，监测节点由工业物联终端设备接入若干传感器组成，搭载 XiUOS，负责实时采

集环境数据并上传到网关节点。网关与监测节点之间采用 LoRa 网络通信，网关节点则负责接收环境数据并转储到分布式内存数据库和分布式磁盘数据库中，供上层应用使用。该系统基于 XiUOS 的多任务管理，并发地管理多个主动上报和被动应答的传感器；基于 XiUOS 的感知框架，简化应用编程，轻松采集工业环境数据。

第 5 章 架构

面向智能制造的 XiUOS 矽璘工业物联网操作系统通过高效管理工业物联网设备、支撑工业物联网应用，实现智能化的全面感知、泛在互联、实时认知、精准调控，促进以工业设备和工业控制系统为核心的人、机、物深度互联，帮助提升生产线的数字化和智能化水平。整体设计方案如图 2-5-1 所示。

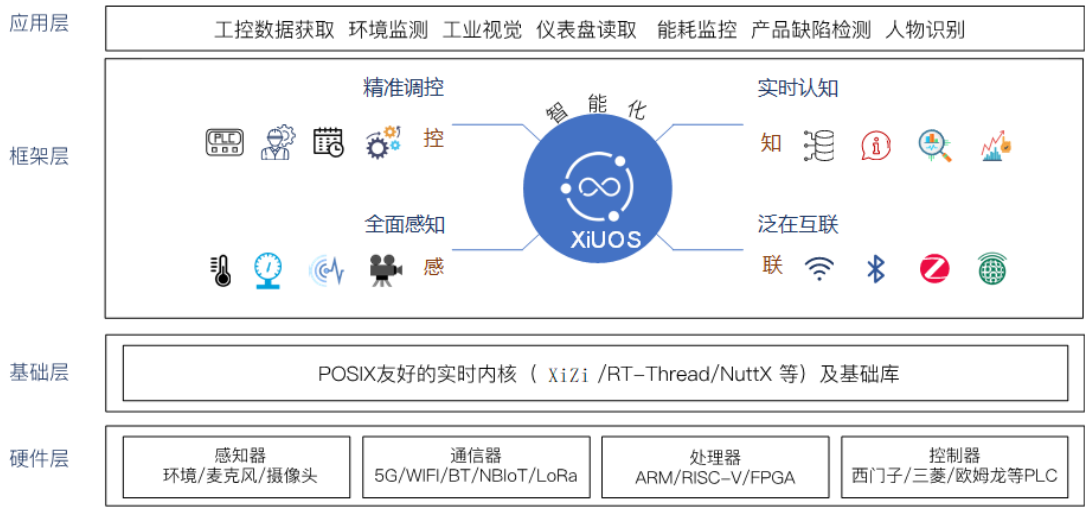


图 2-5-1 XiUOS 系统架构图

（1）硬件层：

硬件层主要是 XiUOS 支持的处理器、感知器和通信器，以及工业领域控制器：

- 处理器: 支持 ARM Cortex-M 系列和 RISC-V 架构的处理器，以及 FPGA 芯片。
- 通信器: 支持 5G/4G/WiFi/BlueTooth/NB-IoT/LoRa 等的网络通信器件。
- 感知器: 支持工业领域的各种环境感知传感器，如烟雾、气体、噪声、电流电压等，还支持对声音和物体进行感知识别的摄像头、麦克风等音视频设备。
- 控制器: 主要 PLC 厂商如西门子、施耐德、欧姆龙、三菱、台达等的 PLC 系统，XiUOS 可从 PLC 中获取工业生产相关数据。

自主研发的硬件系统对感知器、通信器、处理器和控制器提供综合支持，具有如下特点：

- XiUOS 的硬件系统支持 RISC-V 和 ARM 两种体系结构，强调 RISC-V 和 ARM 在工业应用中的共生共赢，又注重突出各自特点；
- XiUOS 的硬件系统在同一物理实体终端上同时支持 4G、NB-IoT、WiFi、ZigBee、LoRa、BlueTooth、Ethernet，在硬件上使得现有应用能够方便、快捷地联接各种网络；
- XiUOS 的硬件系统在同一物理实体终端上同时支持 SPI、IIC、UART、CAN、RS485、RS232、USB、SDIO 等总线接口，在硬件上满足工业物联网对外设多样性的需求；
- XiUOS 的硬件系统能够直接部署于工业生产环境中，而非验证性的开发板评估板，系统稳定可靠。

XiUOS 硬件系统中的处理器、通信器、传感器、控制器采用解耦的模块化设计，基于软件定义思想屏蔽各种硬件外设的差异，使得工业物联网用户不必关心底层硬件的实施细节，从而更加专注于应用逻辑本身的效能。

（2）基础层：

基础层是 XiUOS 关于实时内核的定义，支持自研 XiZi 内核，提供稳定的内核服务和驱动抽象。内核服务提供线程管理、内存管理、文件系统、中断管理和节拍机制等核心服务，支持多线程运行。驱动抽象框架遵循外设总线化和文件化的中心思想，驱动和应用解耦，支持海量异构的硬件设备。同时为拓展生态、吸引开发者，兼容 POSIX 友好的开源 RTOS 内核，如 RT-Thread 和 NuttX。

从 XiUOS 整体架构角度，基础层南向支持硬件层各类工业物联网设备的驱动，北向对接框架层“感、联、知、控”应用框架，除实时内核外，还提供丰富基础库，如 C/C++标准库、轻量级网络协议栈 LwIP、Json 文件解析库 cJson、嵌入式图形库 LittleVGL、物联网通信协议 MQTT 等。

基于对现有内核的功能特性与参数的归纳抽象，支持对内核进行软件定义，用户可选择基于何种内核运行 XiUOS，并对公共库、驱动等进行相应的定义，根据配置进行加载或生成。

（3）框架层：

框架层面向工业物联场景进行设计，核心是“感、联、知、控”应用框架，具体描述如下：

- 感：支持工业领域的各种环境感知传感器，如温度、湿度、烟雾、气味、噪声、电流电压等，以及对声音和物体进行感知识别的摄像头等音视频采集设备，提供方便各种外接接口驱动，如 IIC、UART 等，便于接入新传感器。同时鉴于每种传感器在底层使用上都有区别，为了简化编程，提供统一的面向物理量的编程抽象，提供统一的面向数据的编程抽象，屏蔽底层设备的使用差异，开发者只需要使用 open/write/read/ioctl/等接口就可以对各种传感器进行操作和开发，简化传感数据的采集、解析和管理。
- 联：互联框架提供网络互联能力，尤其是复杂环境下的无线自组网能力，支持面向嵌入式领域的各种网络协议栈和消息通信库。其中协议栈包括 TCP/IP、NB-IoT、LoRa 等的软件栈支持，方便开发者开发通信功能。消息通信库则包括 MQTT、CoAP 等常用的 IoT 通信库。设计上采用以网络适配器为中心的抽象方式，发送与接收数据时，只需要像 socket 网络通信一样，仅关注发送与接收的数据，而不用把重点放在所使用的具体协议上，有效屏蔽种类繁多的网络协议细节，对外提供统一的网络接口，从而实现了网络数据收发功能与底层硬件的解耦，底层驱动实现具体的网络协议，从而将上层的网络数据传送需求映射到具体的硬件通信模组上。
- 控：控制框架提供工控设备控制和数据采集能力，提供对 PLC 的数据采集库，可对设备进行实时监控和远程控制，遵循软件定义工业控制的设计思路，支持和常见 PLC、工控机设备的互联互通，建立信息控制和交互的通道，实现自顶向下的执行流。
- 知：智能框架是 XiUOS 的一个特色，将智能识别和分析能力下沉到物联网节点设备上(通常为带加速器的 ARM Cortex-M/A 或 RISC-V 处理器)，使之在节点端即可实时对声音、物体、人体、缺陷模式等进行识别，大大拓展物联网节点的应用范围，内容包括 AI 算法加速库、微型 AI 引擎等。其中 AI 算法加速库提供工业物联网方面的常用 AI 模型，比如仪表盘识别模型等，微型 AI 引擎提供运行环境，如 TensorFlow Lite for mircocontrollers，方便开发者开发常用 AI 算法。此外，还根据处理器和加速器的具体情况，提供加速器驱动，来高效利用节点端算力。

第 3 章 XiZi 内核原理

实时内核是一个实时操作系统的核心和基础所在，其对下支持外设资源的抽象，对上提供高效实时的内核对象，从而满足实时操作系统对实时应用业务的支撑。XiUOS 支持多种实时内核，即自研 XiZi 内核、国产操作系统内核 RT-thread、国外嵌入式操作系统 NuttX，本章将从自研 XiZi 内核角度进行阐述，XiZi 内核由任务线程、内存、中断、时钟、文件系统和驱动模型等模块组成，如图 3-1 所示，

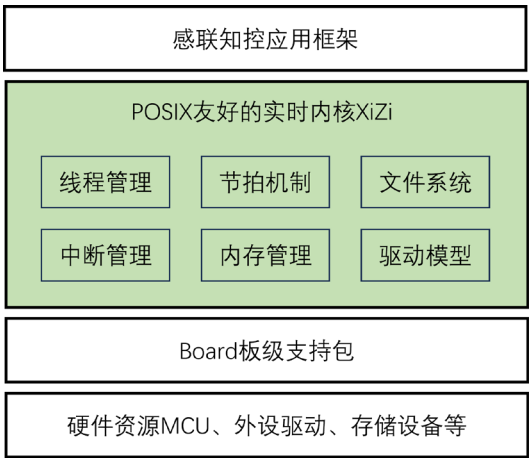


图 3-1 XiZi 内核示意图

3.1 线程管理

任务线程 Task 是 XiZi 内核中处理器 MCU 使用权分配的最小单位，支持单核和多核处理器。线程有自己的程序栈、寄存器上下文、属性，线程之间互不影响，各自处理各自业务逻辑，XiZi 内核中线程支持优先级配置（支持 32 个优先级配置，0 为优先级最低，设置为空闲 Idle 线程），重要线程可设置高优先级，一般线程可设置相同优先级，平分处理器资源。由线程调度器根据抢占式机制和时间片轮转算法调度线程运行、挂起，即优先级高的线程可抢占优先级低的线程运行，相同优先级的线程则根据分配的时间片运行，避免出现单个线程独占处理器资源，阻塞系统运行，实现多线程共享处理器资源，从而满足内核线程运行的实时性。内核中默认开启多个系统线程，如空闲线程 Idle、回收线程 ZombieRecycleKTask、主线程 Main、人机交互线程 Lettle-shell 等，除此之外用户可自由创建、删除应用线程，以满足实时任务响应的需求。

本节将从线程数据结构、线程状态切换、线程通信对象、线程隔离机制和线程管理接口等部分进行介绍。

3.1.1 线程数据结构

XiZi 内核线程由 kernel/thread 目录内文件实现，主要包括线程 ktask.c、调度器 assign.c，相关数据结构由头文件 xs_ktask.h、xs_assign.h 定义。

I 线程结构体 TaskDescriptor：

每个线程由一个线程结构体 TaskDescriptor 描述，其结构体定义如代码 3-1-1 所示。

代码 3-1-1 TaskDescriptor 结构体

```
01  struct TaskDescriptor
02  {
03      void                        *stack_point;
04      TaskDyncSchedMembeType      task_dync_sched_member;
05      TaskBaseInfoType            task_base_info;
06      #ifdef ARCH_SMP
07      TaskSmpInfoType             task_smp_info;
08      #endif
09      #if defined (KERNEL_EVENT)
10      uint32                      event_id_trigger : 29;
11      uint32                      event_mode : 3;
12      #endif
13      x_err_t                     exstatus;
14      DoubleLinklistType          link;
15      struct IdNode               id;
16      struct KTaskDone            *Done;
17  };
18  typedef struct TaskDescriptor *KTaskDescriptorType;
```

其中，stack_point 指向任务堆栈的起始地址；task_dync_sched_member 结构体为线程调度相关；task_base_info 记录线程的基本信息；task_smp_info 表示多核处理器线程相关信息；event_id_trigger/event_mode 用于实现事件集机制（详见线程通信章节）；exstatus 为线程调用内核接口时错误码，即线程在使用内核接口时可能会执行失败，此时内核接口返回-1，具体的错误码被保存在成员变量 exstatus 中，且在下一次调用内核接口失败时被覆盖；link 为线程管理双向链表；id 用于表示线程号；Done 提供线程的操作回调函数。

II 线程调度结构体 TaskDyncSchedMembeType：

线程调度由结构体 TaskDyncSchedMembeType 实现，由代码 3-1-2 中所示。XiZi 内核的线程调度指对系统中的所有线程按照一定策略（抢占式优先级或先来先服务）进行调度，最大限度地利用处理器资源，即从系统就绪队列中按调度策略选择某一线程，使其进入运行状态。

XiZi 内核线程调度支持抢占式优先级调度，即在就绪队列中，高优先级的线程会抢占低优先级的线程的处理器资源，系统将低优先级线程挂起，转而执行高优先级线程。相同优先级的线程调度则支持以下三种方式：

- 时间片轮转（Round-Robin，RR）调度，即线程按照分配的时间片执行，时间片结束，调度一个新的就绪线程执行，当前线程重新就绪，等待下一次的调度；
- 时间片轮转+剩余时间片（RR_REMAINSLICE）调度，在 RR 调度策略中，每个线程只运行一个剩余时间片，结束后就将该线程放到等待队列中。若要使某个线程运行 n 个剩余时间片，那么采用 RR_ REMAINSLICE 的调度方式，其中剩余时间片可以自定义设置；
- 先进先出（First Input First Output，FIFO）调度，即线程按照被创建的顺序依次被执行。当一个线程运行完成后，系统才会让下一个线程开始运行。

代码 3-1-2 TaskDyncSchedMembeType 结构体

```
01  struct TaskDyncSchedMember {
02      uint8                   stat;
03      uint8                   advance_cnt;
04      uint8                   cur_prio;
05      ubase                    origin_timeslice;
06      ubase                    rest_timeslice;
07  #ifdef SEPARATE_COMPILE
08      uint8                   isolation_flag;
09      void                     *isolation;
10      uint8                   isolation_status;
11  #endif
12      DoubleLinklistType      sched_link;
13  #if KTASK_PRIORITY_MAX > 32
14      uint8   bitmap_offset;
15      uint8   bitmap_row;
16  #endif
17      uint32                    bitmap_column;
18      delay_t                   delay;
19  };
20  typedef struct TaskDyncSchedMember TaskDyncSchedMembeType;
21
22  #define KTASK_INIT                  0x00
23  #define KTASK_READY                 0x01
24  #define KTASK_SUSPEND               0x02
25  #define KTASK_RUNNING               0x03
26  #define KTASK_CLOSE                 0x04
```

其中，stat 记录线程的当前状态，可设置为初始化状态（KTASK_INIT）、挂起状态（KTASK_SUSPEND）、就绪状态（KTASK_READY）、运行状态（KTASK_RUNNING）或退出状态（KTASK_CLOSE）；advance_cnt 表示在调度时优先处理的时间片周期个数；cur_prio 表示线程当前的优先级；origin_timeslice 表示在时间片轮转调度时，线程每次运行的时间片；rest_timeslice 表示在时间片轮转调度时，线程运行剩余时间片；在用户态、内核态分开编译时，isolation_flag 变量和指针 isolation 支持地址空间隔离，isolation_status 用于标志内核服务的过程（1 表示进入内核服务上下文）；bitmap_column 为 32 位，用于记录线程优先级，即 0-31 位某一位置 1 表示该线程的优先级；sched_link 表示就绪队列双向链表节点；task_timer 为线程延时计数器。

III 线程基本信息结构体 TaskBaseInfoType：

TaskBaseInfoType 结构记录了线程的基本属性。线程创建时，根据需求可配置该结构体参数，设置合适的线程栈大小、线程入口函数、线程优先级等，其中线程栈大小取决于 MCU 硬件内存大小和线程业务复杂程度。结构体具体如代码 3-1-3 所示。

代码 3-1-2 TaskBaseInfoType 结构体

```
01  struct TaskBaseInfo{
02          char          name[NAME_NUM_MAX];
03          void          *func_entry;
04          void          *func_param;
05          uint8          origin_prio;
06          uint32         stack_depth;
07          void          *stack_start;
08  };
09  typedef struct TaskBaseInfo TaskBaseInfoType;
```

其中，name 表示线程的名称；func_entry 表示线程入口函数；func_param 表示线程的传递参数；origin_prio 表示线程初始优先级；stack_depth 表示线程的栈大小，该线程内的局部变量均从线程栈空间申请，同时发生线程切换时，线程上下文信息均保存到线程栈中，待重新切换回来时则从线程栈中恢复上下文信息，重新执行线程；stack_start 表示线程栈地址指针。

IV 线程就绪队列数据结构 OsAssignReadyVector：

OsAssignReadyVector 记录了线程就绪队列配置信息，其结构体如代码 3-1-4 所示。

代码 3-1-4 OsAssignReadyVector 结构体

```
01  struct OsAssignReadyVector {
02          DoubleLinklistType priority_ready_vector[KTASK_PRIORITY_MAX];
03          uint32 priority_ready_group;
04          #if KTASK_PRIORITY_MAX > 32
05          /* Maximum priority level, 256 */
06          #endif
07          x_ubase highest_prio;
08  };
```

其中，priority_ready_vector 为线程就绪队列，由双向链表实现，最大个数为内核优先级大小，默认为 32；priority_ready_group 为 32 位数，每一位记录就绪队列中线程优先级；highest_prio 记录就绪队列中所有线程的最高优先级。

3.1.2 线程状态切换

XiZi 内核中的线程在任意时刻都处于就绪（Ready）、运行（Running）、阻塞/挂起（Suspend）、退出（Close）四种状态之一。状态之间的变化关系如下图所示。

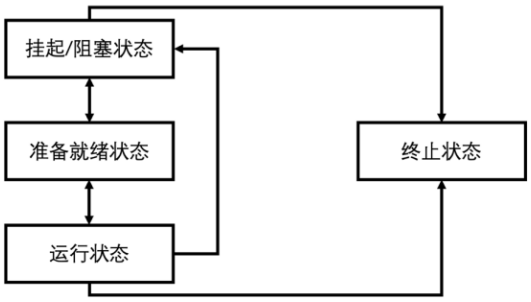


图 3-1-1 线程状态切换图

- 线程在创建完成后会进入就绪状态，并被加入就绪队列等待内核 CPU 调度；
- 当线程被调度开始运行时，会进入运行状态；
- 如果线程在运行过程中被更高优先级的线程抢占，则被抢占的线程会回到就绪队列，并再次进入就绪状态；
- 当线程在运行过程中申请资源失败时，线程会被挂起到对应资源的挂起队列，如信号量、事件、互斥锁、队列等内核对象，并进入挂起状态。当所申请的资源能够被满足时，会回到就绪状态；
- 当线程执行完成，即从入口函数返回时，会进入终止状态，并由内核回收其相关资源。

3.1.3 线程通信对象

XiZi 内核中提供多种线程间通信、同步机制，支持线程间数据交互、时序同步等功能。通信对象主要包括消息队列，同步对象主要包括信号量、互斥量、事件集。

I 消息队列：

消息队列（message queue）提供可供多个线程读或写的消息缓冲区，其中消息指固定长度的任意数据块。消息队列的容量有限，当消息队列满时，向消息队列写入的线程会被挂起；当消息队列空时，从消息队列读取的线程会被挂起，直到等待的资源满足时线程会重新放入就绪队列，等待调度执行。消息队列的结构定义及成员含义如下所示：

代码 3-1-5 消息队列结构定义

```
01  struct MsgQueue{
02  struct IdNode          id;              //用于唯一标识一个消息队列
03  void                   *msg_buf;        //用于存储消息的缓冲区的首地址
04  int16                  index;           //当前缓冲区中第一个消息的序号
05  uint16                 num_msgs;        //当前缓冲区中的消息数量
06  uint16                 each_len;        //每条消息的大小，单位为字节
07  uint16                 max_msgs;        //缓冲区中最多能存放的消息数量
08  DoubleLinklistType     send_pend_list;  //被挂起的发送任务链表
09  DoubleLinklistType     recv_pend_list;  //被挂起的接收任务链表
10  DoubleLinklistType     link;            //系统中所有消息队列构成的链表
```

```
11          struct MsgQueueDone *Done;          //消息队列的操作函数集
12  };
```

XiZi 内核中定义了一系列消息队列函数接口：

表 3-1-1

函数原型	**int32_t KCreateMsgQueue (size_t msg_size, size_t max_msgs)**
函数功能	创建一个消息队列，创建成功后，新的消息队列会被加入内核的消息队列管理链表，并返回该消息队列的 ID，ID 默认范围 0-255，可配置
参数说明	参数 msg_size 表示每条消息的长度，单位为字节；max_msgs 表示缓冲区中最多存放的消息数量
返回值	返回所创建消息队列的 ID，若创建失败则返回-ENOMEMORY

表 3-2-2

函数原型	**x_err_t KDeleteMsgQueue (int32_t mq)**
函数功能	删除一个已创建的消息队列
参数说明	参数 mq 表示待删除的消息队列 ID
返回值	返回 EOK

表 3-3-3

函数原型	**x_err_t KMsgQueueSendwait (int32_t mq, const void *buffer, size_t size, int32_t wait_time)**
函数功能	等待一段时间后，再向消息队列发送一个消息
参数说明	参数 mq 表示目标消息队列 ID；buffer 表示消息数据首地址；size 表示消息长度；wait_time 表示等待时间上限，单位 ms，若为 0，则不等待，若为-1，表示永久等待
返回值	若消息发送成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT

表 3-4-4

函数原型	**x_err_t KMsgQueueSend (int32_t mq, const void *buffer, size_t size)**
函数功能	向消息队列发送一个消息
参数说明	参数 mq 目标消息队列 ID；buffer 表示消息数据首地址；size 表示消息长度
返回值	若消息发送成功则返回 EOK，若不成功则返回-ETIMEOUT

表 3-5-5

函数原型	**x_err_t KMsgQueueUrgentSend (int32_t mq, const void *buffer, size_t size)**
函数功能	向消息队列优先发送一个消息
参数说明	参数 mq 目标消息队列 ID；buffer 表示消息数据首地址；size 表示消息长度
返回值	若消息发送成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT

表 3-6-6

函数原型	**x_err_t KMsgQueueRecv (int32_t mq, void *buffer, size_t size,int32_t wait_time)**

函数功能	从消息队列接收一个消息
参数说明	参数 mq 表示来源消息队列 ID；buffer 表示用于接收消息数据的缓冲区；size 表示缓冲区大小；wait_time 表示等待时间上限，单位 ms；若为 0，则不等待
返回值	若消息接收成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT

表 3-7-7

函数原型	x_err_t KMsgQueueReinit (int32_t mq)
函数功能	将一个消息队列复位
参数说明	参数 mq 表示消息队列 ID
返回值	返回 EOK

II 信号量：

信号量（semaphore）是实现线程间同步与互斥的内核对象，创建时具有一个初值，线程可以获取或释放一个信号量。当线程获取信号量时，信号量的值递减，释放信号量时，信号量的值递增。当信号量的值递减至 0 时，后续尝试获取信号量的线程会被挂起，当线程释放信号量时，内核会从该信号量的挂起队列上唤醒一个任务。信号量结构定义及成员含义如下所示：

代码 3-1-6 信号量结构定义

```
01  struct Semaphore{
02      struct IdNode       id;         //信号量 ID，用于唯一标识一个信号量
03      uint16              value;      //信号量的当前值
04      DoubleLinklistType  pend_list;  //挂起任务链表
05      DoubleLinklistType  link;       //系统中所有信号量构成的链表
06  };
```

XiZi 内核定义了一系列信号量函数接口：

表 3-8-8

函数原型	sem_t KSemaphoreCreate (uint16_t val)
函数功能	创建一个信号量，创建成功后，新的信号量会被加入内核的信号量管理链表，并返回该信号量的 ID，ID 默认范围 0-255，可配置
参数说明	参数 val 表示信号量的初值
返回值	返回所创建信号量的 ID，若创建失败则返回-ENOMEMORY

表 3-9-9

函数原型	x_err_t KSemaphoreDelete (sem_t sem)
函数功能	删除一个已创建的信号量
参数说明	参数 sem 表示待删除的信号量 ID
返回值	返回 EOK

表 3-10-10

函数原型	x_err_t KSemaphoreObtain (sem_t sem, int32_t wait_time)
函数功能	获取一个信号量
参数说明	参数 sem 表示待获取的信号量 ID；wait_time 表示等待时间上限，单位 ms，若为 0，则不等待
返回值	若获取成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT

表 3-11-11

函数原型	x_err_t KSemaphoreAbandon (sem_t sem)
函数功能	释放一个信号量
参数说明	参数 sem 表示待释放的信号量 ID
返回值	返回 EOK

表 3-12-12

函数原型	x_err_t KSemaphoreSetValue (sem_t sem, uint16_t val)
函数功能	将一个信号量的值进行重置
参数说明	参数 sem 表示来源消息队列 ID；val 表示重置的信号量的值
返回值	返回 EOK

III 互斥量：

互斥量（mutex）是实现线程间共享资源互斥的内核对象，仅有两种状态：锁和解锁。线程拥有互斥量时，处于锁状态，其余线程不可拥有其所有权，挂起等待互斥量释放；线程释放互斥量时，处于解锁状态，其他线程可拥有其所有权，重新加入就绪队列。互斥量结构定义及成员含义如下所示：

代码 3-1-7 互斥量结构定义

```
01  struct Mutex{
02      struct IdNode          id;             //互斥量 ID，用于唯一标识一个互斥量
03      uint16                 val;            //互斥量的当前值
04      uint8                  recursive_cnt; //持有互斥量的任务的原优先级
05      uint8                  origin_prio;   //持有互斥量的任务获取互斥量的次数
06      struct TaskDescriptor *holder;         //持有互斥量的任务
07      DoubleLinklistType     pend_list;      //挂起任务链表
08      DoubleLinklistType     link;           //系统中所有互斥量构成的链表
09  };
```

XiZi 中定义了一系列互斥量函数接口：

表 3-13-13

函数原型	int32_t KMutexCreate ()
函数功能	创建一个互斥量。创建成功后，新的互斥量会被加入内核的互斥量管理链表，并返回该互斥量的 ID，ID 默认范围 0-255，可配置

参数说明	无
返回值	返回所创建互斥量的 ID，若创建失败则返回-ENOMEMORY

表 3-14-14

函数原型	void KMutexDelete (int32_t mutex)
函数功能	删除一个已创建的互斥量
参数说明	参数 mutex 表示待删除的互斥量的 ID
返回值	无

表 3-15-15

函数原型	int32_t KMutexObtain (int32_t mutex, int32_t wait_time)
函数功能	获取一个互斥量
参数说明	参数 mutex 表示待获取的互斥量的 ID；wait_time 表示等待时间上限，单位 ms；若为 0，则不等待
返回值	若获取成功则返回 EOK，若不成功（等待超时）则返回-ETIMEOUT

表 3-16-16

函数原型	int32_t KMutexAbandon (int32_t mutex)
函数功能	释放一个互斥量
参数说明	参数 mutex 表示待释放的互斥量的 ID
返回值	若释放成功返回 EOK，若不成功则返回-ERROR

IV 事件集：

事件集（event）允许用户定义一个事件集合，集合中的每个事件都可以被线程触发或等待。线程可以同时等待多个事件，此时等待触发的条件可以配置为 AND 或者 OR：当等待触发条件配置为 AND 时，只有所有被等待的事件均被触发才视作等待结束，称之为逻辑与触发；当等待触发条件配置为 OR 时，任意一个被等待的事件触发即视作等待结束，称之为逻辑或触发。使用事件集可以实现多对多的线程间同步与互斥。事件集结构定义及成员含义如下所示：

代码 3-1-8 事件集结构定义

```
01  #define EVENT_AND              (1 << 0)
02  #define EVENT_OR               (1 << 1)
03  #define EVENT_AUTOCLEAN        (1 << 2)
04  #define EVENTS(x)              (1 << x)
05  struct Event{
06      struct IdNode          id;           //用于唯一标识一个事件集
07      uint32                 options : 3;  //高 3 位用于记录事件集属性
08      uint32                 events : 29;  //低 29 位用于表示至多 29 个事件
09      DoubleLinklistType     pend_list;    //等待任务链表
10      DoubleLinklistType     link;         //系统中所有事件集构成的链表
11  };
```

XiZi 内核中定义了一系列事件集函数接口：

表 3-17-17

函数原型	eventIdType KEventCreate (uint8_t flag)
函数功能	创建一个事件集，创建成功后，新的事件集会被加入内核的事件集管理链表，并返回该事件集的 ID，ID 默认范围 0-255，可配置
参数说明	参数 flag 用于配置事件集的属性。可配置的属性有等待触发方式（EVENT_AND 或 EVENT_OR）及等待触发后是否自动清空其他已触发的事件（EVENT_AUTOCLEAN）
返回值	返回所创建事件量的 ID，若创建失败则返回-ENOMEMORY

表 3-18-18

函数原型	void KEventDelete (EventIdType event)
函数功能	删除一个已创建的事件集
参数说明	参数 event 表示待删除的事件集的 ID
返回值	无

表 3-19-19

函数原型	x_err_t KEventTrigger (EventIdType event, uint32_t set)
函数功能	触发事件集中的一组事件
参数说明	参数 event 表示事件集 ID；set 表示欲触发的事件，其中被置 1 的位标识被触发的事件,可以使用 EVENTS 宏按位或发送事件类型
返回值	若触发成功则返回 EOK，若不成功则返回-ERROR

表 3-20-20

函数原型	x_err_t KEventProcess (EventIdType event, uint32_t set, uint8_t option, int32_t wait_time, uint32_t *Recved)
函数功能	等待事件集中的一组事件
参数说明	参数 event 表示事件集 ID；set 表示欲等待的事件，其中被置 1 的位标识被等待的事件，可以使用 EVENTS 宏按位或接收事件类型；options 表示用于记录事件集属性（等待触发条件、是否自动清空）；wait_time 表示等待时间上限，单位 ms；若为 0，则不等待；Recved 表示用于记录已经被处理完成的事件
返回值	若等待成功则返回 EOK，此时若 EVENT_AUTOCLEAN 被打开则事件集中所有已触发事件会被清空；若等待失败（超时）则返回-ETIMEOUT

3.1.4 线程隔离机制

XiUOS 是一个支持多线程的操作系统，对线程的数量没有限制，在 XiZi 内核中，每个线程都需要自己的堆栈，同时也可能会动态申请内存资源。线程在运行过程中发生内存溢出是实时操作系统（RTOS）中最常见的问题，所以限制线程的内存空间访问是保证 RTOS 稳定运行的关键。

ARM 和 RISC-V 在体系架构上都提供了内存访问的保护功能，可以通过对特定寄存器的硬编程实现对指定内存区域访问权限的设置，即 ARM 架构的 MPU（Memory Protection

Unit）、RISC-V 架构的 PMP（Physical Memory Protection）。然而，现有的大多数物联网操作系统并没有使用体系结构提供的内存保护功能来对线程运行的地址空间进行隔离保护。XiZi 内核充分考虑线程运行的安全问题，在不影响线程正常执行的情况下，对每个线程所允许访问的内存地址空间进行限制。除此之外，线程在动态申请内存、释放内存、内存共享时，XiZi 内核也将提供隔离服务。

线程隔离机制的总体设计思想是将物理内存地址空间划分为信任地址空间和非信任地址空间。XiZi 的内核线程运行特权模式和信任地址空间，可以访问所有信任地址空间和非信任地址空间；XiZi 的用户程序运行在非特权模式和非信任地址空间，通过“内核服务表 g_service_table[256]”的方式访问内核提供的功能和对象，其中“内核服务表 g_service_table[256]”指为用户程序提供一组服务接口来满足应用程序的这些需求，访问内核服务接口的流程如下：

- 应用程序执行异常调用指令并指定相应内核服务号和参数；
- 通过软中断指令产生一个调用异常，之后 CPU 切换到特权模式并强制执行异常处理流程，异常处理流程提取内核服务号和参数，并将服务号作为索引；
- 根据索引从内核服务表中查找对应的内核服务接口；
- 在特权模式下，执行所需的内核服务，完成后切换回非特权模式继续执行。

XiZi 内核中通过描述符 TaskDescriptorType 管理系统的线程，其中包含了任务优先级、线程名称、线程状态等信息。为了管理线程可访问的内存空间，TaskDescriptor 描述符在 TaskDyncSchedMember 子结构中增加任务隔离标志位 isolation_flag 和任务隔离成员 isolation、isolation_status，其定义如下：

代码 3-1-9 TaskDyncSchedMember 结构

```
01  struct TaskDyncSchedMember {
02              …
03      #ifdef SEPARATE_COMPILE
04              uint8           isolation_flag;
05              void           *isolation;
06              uint8           isolation_status;
07      #endif
08              …
09  };
```

用户程序运行在非信任地址空间，默认开启隔离机制，即 SEPARATE_COMPILE 宏使能。在用户程序对应的线程被创建时，线程隔离标志位 isolation_flag 被置为 1，其允许访问的地址空间范围由 isolation 指针所指定。isolation 数据结构包括一段连续的地址空间范围和访问权限，其具体结构取决于体系架构提供的内存保护功能，如 ARM 提供的 MPU 和 RISC-V 提供的 PMP 功能。isolation 的定义如下：

代码 3-1-10 isolation 定义

```
01  #ifdef TASK_ISOLATION
02
03  #if defined(ARCH_ARM) && defined(SURPORT_MPU)
04
05  struct Mpu *isolation = NONE;
06  struct MemoryAccessLimit mem_access = {
```

```
07          .Enable = MpuEnable ,
08          .Disable = MpuDisable ,
09          .Init = MpuInit,
10          .InitIsolation = NONE,
11          .AddRegion = NONE,
12          .ClearRegion = NONE,
13          .Free = NONE,
14          .Load = MpuLoad,
15          .FaultHandle = NONE,
16  };
17  #endif
18
19  #if defined(ARCH_RISCV) && defined(SURPORT_PMP)
20
21  struct Pmp *isolation = NONE;
22  struct MemoryAccessLimit mem_access = {
23          .Enable = NONE,
24          .Disable = NONE,
25          .Init = NONE,
26          .InitIsolation = PmpInitIsolation,
27          .AddRegion = PmpAddTorRegion,
28          .ClearRegion = PmpClearRegion,
29          .Free = PmpFree,
30          .Load = PmpLoad,
31          .FaultHandle = PmpAccessFaultHandle,
32  };
33
34  #endif
35
36  #endif
```

XiZi 内核根据 ARM MPU 和 RISC-V PMP 机制，分别定义了两套 isolation 数据结构，对应两套内存操作 mem_access 数据结构，mem_accsss 中定义了 Enable、Disable、Init、InitIsolation、AddRegion、ClearRegion、Free、Load、FaultHandle 回调函数，对接 MPU 或 PMP 中寄存器操作。

基于上述的任务内存结构分布及 isolation 数据结构，下面介绍面向 RISC-V 架构的三种线程隔离机制：线程切换时的隔离、动态申请/释放内存时的隔离、RISC-V 64 架构线程隔离。

（1）线程切换时的隔离：

在 XiZi 内核中，多个线程共享有限个 CPU 核，采用优先级加时间片的调度方式，当一个线程时间片耗尽或者主动让出 CPU 的使用权时，内核调度程序负责保存当前线程的上下文信息，并从等待队列中挑选下一个就绪线程，恢复其上下文信息，并允许其获取 CPU 的使用权。为了保证每个线程只能访问自己的内存空间，内核调度程序在恢复线程上下文时，会先根据 isolation_flag 标志判断该线程是否为用户程序，如果为用户程序，则当前 CPU 核在运行该线程时，只允许访问本任务 AddRegion 中定义的内存区域，对于其它区域没有访问权限。

（2）动态申请/释放内存时的隔离：

XiZi 内核的线程通过 malloc/free 等内核服务接口来动态申请和释放内存空间，并能根据用户线程申请和释放的内存地址更新线程隔离表。当用户程序通过 malloc 等内核服务接口向内核申请指定大小的内存空间时，系统会在用户程序对应的线程中增加对内存空间的访问权限，同时将更新后的线程隔离表加载到内存保护单元配置寄存器 MPU 或 PMP 中，使其生效。当用户程序通过 free 等内核服务接口向内核释放指定大小的内存空间时，系统会在用户程序对应的线程隔离表中清除对这段内存空间的访问权限，同时加载更新后的任务隔离表到内存保护单元寄存器 MPU 或 PMP 中，使其生效。

此外，XiZi 内核还支持共享内存的线程隔离，其基本思路同动态内存申请/释放时相同。进一步地，当用户程序因为某种原因试图访问没有权限的内存空间时，CPU 会产生一个访问错误的异常，并进入内核服务接口的异常处理流程，在这个流程中会将该任务直接杀死并回收线程资源。

（3）RISC-V 64 架构线程隔离：

大部分 RISC-V 架构的 CPU 拥有特权模式和非特权模式。这两种模式都为 CPU 核提供了物理内存保护 PMP 功能。通过编程 PMP，可以为指定大小的内存区域设置读、写、执行等访问权限。PMP 包括 8~16 组配置寄存器和地址寄存器，一组配置寄存器和地址寄存器称为一个 PMP entry，PMP entry 用于标识一段内存地址空间的访问权限。

3.1.5 线程管理接口

XiZi 内核中定义了一系列线程函数接口，用于线程创建、删除、延时等。线程的各个属性由 utask 结构表示，包括线程的名称、入口函数及参数、栈大小和优先级，在调用该函数时需要传入该结构的实例用于配置线程属性，线程创建成功后，内核会为其分配指定大小的栈及其他结构（如 struct TaskDescriptor），并返回线程 id。线程管理相关的函数接口定义如下：

表 3-21-21

函数原型	**int32_t KTaskCreate (const char *name, void (*entry)(void *parameter), void *parameter, uint32 stack_depth, uint8 priority)**
函数功能	创建一个线程
参数说明	参数 name 是线程的名字；参数 entry 是线程的入口函数；参数 parameter 是线程入口函数 entry 的传参；参数 stack_depth 是线程栈的大小；参数 priority 是线程的优先级
返回值	若创建成功返回线程的 ID，若创建失败，线程内存申请失败返回 -ENOMEMORY，其余情况则返回-ERROR

表 3-22-22

函数原型	**x_err_t StartupKTask (int32_t id)**
函数功能	就绪一个线程，交由调度器开始调度执行
参数说明	参数 id 表示待就绪的线程 ID
返回值	若成功返回 EOK，若失败，线程 ID 无效返回-EINVALED，其余情况则返回 -ERROR

表 3-23-23

函数原型	x_err_t KTaskDelete (int32_t id)
函数功能	删除一个线程，强制其进入退出状态
参数说明	参数 id 表示待删除的线程 ID
返回值	若删除成功返回 EOK，若失败，线程 ID 无效返回-EINVALED，其余情况则返回-ERROR

表 3-24-24

函数原型	x_err_t DelayKTask (x_ticks_t ticks)
函数功能	将当前线程挂起一定时间，单位为 ticks 时间片，挂起时间结束后，线程会进入就绪状态，等待系统调用
参数说明	参数 ticks 表示线程的挂起时间片个数
返回值	若成功返回 EOK，若失败，ms 设置为 0 返回-EINVALED，线程无效则返回-EEMPTY，其余情况则返回-ERROR

表 3-25-25

函数原型	x_err_t MdelayKTask (uint32 ms)
函数功能	将当前线程挂起一定时间，单位为毫秒 ms，挂起时间结束后，线程会进入就绪状态，等待系统调用
参数说明	参数 ms 表示线程的挂起时间
返回值	若成功返回 EOK，若失败，ms 设置为 0 返回-EINVALED，线程无效则返回-EEMPTY，其余情况则返回-ERROR

表 3-26-26

函数原型	x_err_t KTaskPrioSet(int32 id, uint8 prio)
函数功能	设置线程的优先级
参数说明	参数 id 表示线程 ID；参数 prio 表示设置的优先级
返回值	若成功返回 EOK，若失败，线程状态无效返回-EINVALED，其余情况则返回-ERROR

表 3-27-27

函数原型	x_err_t SuspendKTask(int32 id)
函数功能	挂起当前线程
参数说明	参数 id 表示线程 ID
返回值	若成功返回 EOK，若失败则返回-ERROR

表 3-28-28

函数原型	x_err_t KTaskWakeup(int32 id)
函数功能	唤醒当前线程，将其重新放入就绪队列中
参数说明	参数 id 表示线程 ID
返回值	若成功返回 EOK，若失败则返回-ERROR

表 3-29-29

函数原型	x_err_t KTaskCoreCombine (int32_t id, uint8_t core_id)
函数功能	将线程绑定至指定的处理器上
参数说明	参数 id 表示待绑定的线程 ID；core_id 表示待绑定的处理器 ID
返回值	若绑定成功则返回 EOK，若失败，线程状态无效返回-EINVALED，其余情况则返回-ERROR

表 3-30-30

函数原型	x_err_t KTaskCoreUnCombine (int32_t id)
函数功能	解除线程与处理器的绑定
参数说明	参数 id 表示待解除绑定的线程 ID
返回值	若解除成功则返回 EOK，若失败，线程状态无效返回-EINVALED，其余情况则返回-ERROR

3.2 内存管理

内存通常分为 RAM 和 ROM，其中 RAM 为随机存取存储器，CPU 可直接读取数据，读写速度较快，容量较小，掉电后数据丢失，也称之为运行内存；ROM 为只读存储器，CPU 通过外设接口读取数据，读写速率相对较慢，容量较大，掉电后数据不丢失，常见的 Flash 即属于 ROM。XiUOS 面向嵌入式设备应用，不同于通用操作系统，如 Linux、Windows，支持 MMU 开启虚拟地址和物理地址映射，XiZi 内核的内存管理针对单地址物理内存空间。嵌入式设备的内存往往较小，RAM 大多在几百 KB 的量级，因此高效、合理、快速地分配、回收内存对于嵌入式操作系统来说非常重要，XiZi 内核的针对物理内存管理主要为内存堆管理，即指对一段连续内存的管理，分为静态内存和动态内存管理。

内存堆可根据线程需要自由申请分配合适大小的内存，并在释放时按一定算法回收合并，可实现分配和释放内存的实时性，提高有限内存的使用率，有效规避内存碎片问题。

本节将从内存数据结构、内存管理机制和内存管理接口等部分进行介绍。

3.2.1 内存数据结构

XiZi 内核内存由 kernel/memory 目录内文件实现，主要包括内存堆 byte_manage.c、内存池 gatherblock.c，相关数据结构由头文件 xs_memory.h 定义。

I 内存堆结构体 ByteMemory：

XiZi 内核内存堆支持静态内存和动态内存管理，前者针对不大于 32 字节或 64 字节的内存申请释放，后者针对任意大小的内存申请释放。内存堆管理由结构体 ByteMemory 描述，其结构体定义如下所示。

代码 3-2-3 ByteMemory 结构体

```
01  struct ByteMemory
```

```
02  {
03      struct DynamicBuddyMemory dynamic_buddy_manager;  /* the manager of
dynamic buddy memory */
04      struct segment static_manager[MM_SMALL_SEGMENTS];     /* the manager of
static memory */
05      struct DynamicAllocNodeDone *done;
06  };
```

其中，dynamic_buddy_manager 表示动态内存管理数据结构；static_manager 表示静态内存管理数据结构，XiZi 内核支持 32 字节和 64 字节两种；done 则指向动态内存申请、释放的回调函数。

II 静态内存结构体 segment：

XiZi 内核静态内存支持 1-256 个 32 字节和 1-128 个 64 字节的小内存块，可满足系统不大于 32 字节或 64 字节的内存申请。静态内存由结构体 segment 描述，其结构体定义如下所示。

代码 3-2-2 segment **结构体**

```
01  struct segment
02  {
03      x_size_t    block_size;     /* record the size of static memory block */
04      uint8 *       freelist;        /* list for all free static memory blocks */
05      int block_total_count;   /* total static memory blocks */
06      int block_free_count;     /* the remaining count of static memory blocks */
07      struct StaticMemoryDone *done;
08  };
```

其中，block_size 表示静态内存块大小的种类，32 字节或 64 字节；freelist 指向空闲静态内存块；block_total_count 表示系统中所有静态内存块的数量；block_free_count 表示系统中空闲静态内存块的数量；done 则指向静态内存申请、释放的回调函数。

III 动态内存结构体 DynamicBuddyMemory：

除去静态内存大小，剩余内存采用动态内存管理，所有内存块由链表管理，可满足系统各模块差异化的内存申请。动态内存由结构体 DynamicBuddyMemory 描述，其结构体定义如下所示。

代码 3-2-3 DynamicBuddyMemory **结构体**

```
01  struct DynamicBuddyMemory
02  {
03      x_ubase dynamic_buddy_start;
04      x_ubase dynamic_buddy_end;
05      x_ubase active_memory;
06      x_ubase max_ever_usedmem;
```

```
07        x_ubase static_memory;
08        uint64 mm_total_size;                                    /* record the total size
of dynamic buddy memory */
09
10        struct DynamicAllocNode *mm_dynamic_start[1];            /* record the start
boundary of dynamic buddy memory */
11        struct DynamicAllocNode *mm_dynamic_end[1];              /* record the end
boundary of dynamic buddy memory */
12        struct DynamicFreeNode mm_freenode_list[MEM_LINKNRS]; /* multiple lists */
13        struct DynamicBuddyMemoryDone *done;
14    };
```

其中，dynamic_buddy_start 用于记录已分配动态内存段起始地址；dynamic_buddy_end 用于记录已分配动态内存段结束地址；active_memory 记录已分配动态内存段大小；max_ever_usedmem 记录已分配动态内存段中最大值；static_memory 表示系统中已分配静态内存块大小；mm_total_size 记录系统可分配动态内存的大小；mm_dynamic_start 记录动态内存头节点地址指针；mm_dynamic_end 记录动态内存尾节点地址指针；mm_freenode_list 记录空闲动态内存块节点，最大数量 MEM_LINKNRS 为 15 个；done 则指向动态内存申请、释放的回调函数。

IV 已分配动态内存节点结构体 DynamicAllocNode：

已分配动态内存节点信息由结构体 DynamicAllocNode 管理，其定义如下所示。

代码 3-2-4 DynamicAllocNode 结构体

```
01    struct DynamicAllocNode
02    {
03        x_size_t size;                 /* the size of dynamicAllocNode */
04        uint32 prev_adj_size;          /* the size of the previous adjacent node, (dynamic
alloc node or dynamic free node */
05        uint32 flag;
06    };
```

其中，size 表示已分配动态内存节点结构体大小；prev_adj_size 表示该节点前一个节点结构体大小，即已分配动态内存节点结构体或空闲动态内存节点结构体大小；flag 表示动态内存种类，即 RAM、扩展 SRAM 等。

V 空闲动态内存节点结构体 DynamicFreeNode：

空闲动态内存节点信息由结构体 DynamicFreeNode 管理，采用双向链表的方式链接，其定义如下所示。

代码 3-2-5 DynamicFreeNode 结构体

```
01    struct DynamicFreeNode
02    {
```

```
03        x_size_t size;
04        uint32 prev_adj_size;          /* the size of the previous adjacent node, (dynamic
alloc node or dynamic free node */
05        uint32 flag;
06
07        struct DynamicFreeNode *next;
08        struct DynamicFreeNode *prev;
09    };
```

其中，size 表示空闲动态内存节点结构体大小；prev_adj_size 表示该节点前一个节点结构体大小，即已分配动态内存节点结构体或空闲动态内存节点结构体大小；flag 表示动态内存种类，即 RAM、扩展 SRAM 等；next 指向下一个空闲动态内存节点指针地址；prev 指向上一个空闲动态内存节点指针地址。

3.2.2 内存管理机制

I 静态内存管理：

静态内存包含 2 个链表，如图所示。静态链表头 1 指向的内存池中存放的静态内存块的大小都是 32 字节，静态链表头 2 所指向的内存池中存放的静态内存块的大小都是 64 字节。此外，系统分别配置了静态链表头 1 和静态链表头 2 中静态内存块的 total_count 个数最大为 256 和 128。因此，静态链表头 1 最多可以响应用户 256 次的小于 32 字节的内存请求，静态链表头 2 最多可以响应用户 128 次的介于 33~64 字节之间的内存请求，一旦对应的静态内存块分配完了，系统会向动态内存区域寻求内存空间分配。

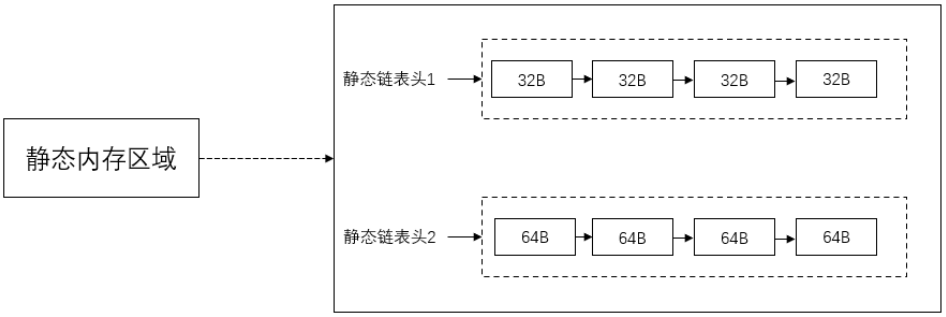


图 3-4-1 静态内存链表

（1）静态内存分配：

在用户发起内存分配请求时，若用户请求分配的内存空间大小小于等于预设的静态内存阈值，且所述静态内存区域有相应空闲的内存块，则从静态内存区域分配相应大小的内存空间。从静态内存区域分配静态内存块的过程，包括三种情况：

- 当用户请求分配内存大小小于等于 32 字节，那么分系统获取静态链表头 1，从静态链表头 1 中获取一个静态内存块返回给用户；
- 当用户请求分配内存大小为 33～64 字节，那么系统获取静态链表头 2，从静态链表头 2 中获取一个静态内存块返回给用户；

- 若所获得的静态链表头是一个空的链表，即没有空闲的静态内存块可用，那么静态内存分配失败。

（2）静态内存释放：

当用户请求释放内存，释放操作也包括三种情况：

- 要释放的内存地址是不合法的，则直接返回释放操作；
- 要释放的内存地址属于静态链表头 1 区域，则将内存块放置到链表头 1 的头部；
- 要释放的内存地址属于链表头 2 区域，则将内存块放置到链表头 2 的头部。

II 动态内存管理：

动态内存区域管理用到了三种重要的数据结构，分别是动态内存管理数据结构 DynamicBuddyMemory、已分配动态内存段 DynamicAllocNode 和空闲动态内存段 DynamicFreeNode。图 3-2-2 显示了已分配动态内存段和空闲动态内存段这两种数据结构。其中，已分配动态内存 DynamicAllocNode 包括元数据信息和用户的数据段（用户数据），元数据信息中的 size 属性记录了该动态内存段的内存大小，prev_size 属性则记录了该动态内存段前一个相邻动态内存段的内存大小，从而可以获取前一个相邻动态内存段。用户数据则是分配给用户使用的内存空间。空闲动态内存段 DynamicFreeNode 的元数据信息具有 size、prev_size、prev 和 next 这四个属性，其中，size 和 prev_size 属性与已分配动态内存段的对应属性表示的意义相同，prev 属性记录了前一个空闲动态内存段，最后一个 next 属性则记录了下一个空闲动态内存段。

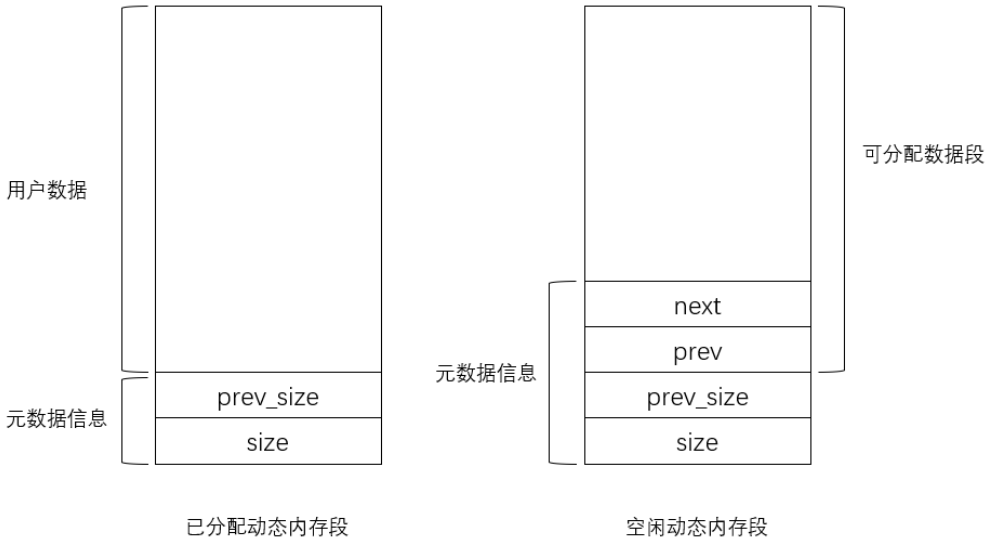


图 3-5-6 空闲动态内存段的元数据信息

图 3-2-3 展示了动态内存区域初始化之后的状态，系统将动态内存区域划分为三个动态内存段，分别是起始地址处已分配动态内存段和结尾已分配动态内存段，中间区域的内存用一个空闲动态内存段记录。

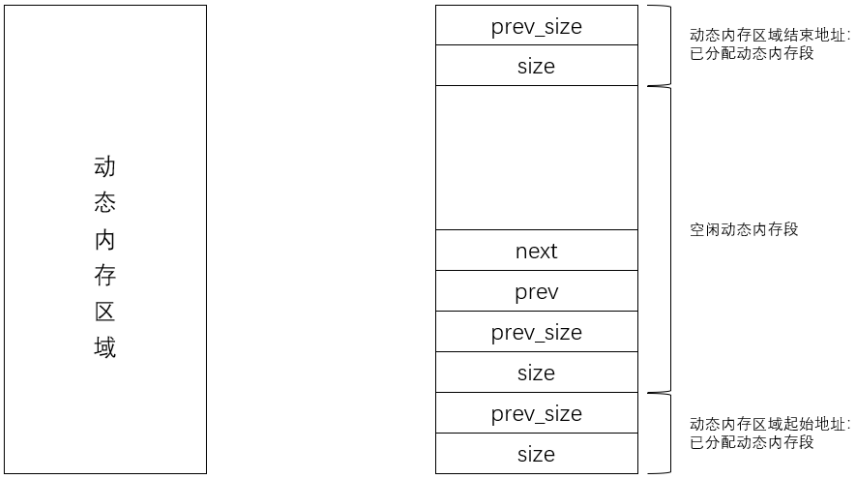


图 3-7-2 动态内存区域记录方式

动态内存管理数据结构DynamicBuddyMemory是动态内存区域管理中的核心数据结构，用于组织所有的空闲动态内存段。所述动态内存管理数据结构具有 total_size、dynamic_buddy_start、dynamic_buddy_end 和 freeLists 这四个属性。其中，total_size 属性用于记录动态内存区域的大小，dynamic_buddy_start 属性用于记录起始地址处已分配动态内存段，dynamic_buddy_end 属性用于记录结尾处已分配动态内存段，freeLists 属性是一个多级链表，每一级链表记录了内存大小在指定范围的空闲动态内存段。根据空闲内存段的大小，系统判断空闲内存段所属动态链表，然后将空闲内存段插入不同的空闲链表中。

系统可配置空闲链表的个数为 15，当动态内存段大小在 64-127 字节之间，则插入到空闲链表 freeList1 中，当动态内存段大小在 128-255 字节之间时，则插入到空闲链表 freeList2 中，需要特别说明的是，当动态内存段大小大于或者等于 524288 字节时，都插入到空闲链表 freeList15 中。如下表所示：

表 3-8-31 动态链表分配

申请的内存范围	分配动态链表头
64~127 字节	freeList1
128~ 255 字节	freeList2
256~511 字节	freeList3
……	……
524288~1048576 字节	freeList15

（1）动态内存分配：

系统根据要分配或者要释放的内存段大小计算要操作的内存段链表头，当内存段大小为 64~127 字节，那么计算之后将获取空闲动态链表头 freeList1，当内存段大小为 128~ 255 字节，那么计算之后将获取动态链表头 freeList2，依次类推。当用户请求从动态内存段申请内存时，有以下几种分配情况：

- 用户请求分配 n 个字节(对齐后的字节)，freeListK 链表中有一个 m 字节内存大小可以满足用户的需求，之后进行内存块的分割操作，分成 n 字节和(n-m)字节，其中 n-m 小于 8 字节，因此将 m 字节内存返回给用户；

- 用户请求分配 n 个字节(对齐后的字节)，freeListK 链表中有一个 m 字节内存大小可以满足用户的需求，之后进行内存块的分割操作，分成 n 字节和（n-m)字节，其中 n-m 大于或等于 8 字节，因此将 n 字节内存返回给用户并且将 n-m 个字节重新插入动态内存管理结构；
- 用户请求分配 n 个字节(对齐后的字节)，freeListK 链表中没有空闲内存大小可以满足用户的需求，并且 freeListK+1 级链表存在，则从 freeListK+1 链表中分配内存；
- 用户请求分配 n 个字节(对齐后的字节)，freeListK 链表中没有空闲内存大小可以满足用户的需求，并且 freeListK+1 级链表不存在，则内存分配失败。

（2）动态内存释放：

当用户请求释放内存，系统释放内存区域到对应的动态内存段中。释放一块动态内存段，下图提供了动态内存段释放和合并的三种情况。其中，每一个动态内存段都包含了两个指针，分别指向前一个相邻内存段和后一个相邻内存段，下面介绍图 3-2-4 展示的释放和合并的三种情况：

- 如图中（a）所示，四块动态内存段，其中，A，B，D 三段为已分配动态内存段，C 为空闲动态内存段。当前，系统要释放内存段 B，动态内存管理模块检测到 B 内存段的后一个相邻 C 也是空闲内存段，则系统会合并内存段 B，C，然后释放 BC 合并之后的内存段。
- 如图中（b）所示，A，B，C，D 四块已分配内存段。系统要释放内存段 B，动态内存管理模块检测到 B 的相邻内存段都是已分配内存段，那么直接释放内存段 B。
- 如图中（c）所示，四块动态内存段，其中，A，C 是空闲内存段；B，D 是已分配内存段。系统要释放内存段 B，动态内存管理模块检测到 B 的前一个相邻内存段 A 和后一个相邻内存段 B 都是空闲内存段，那么先合并 A，B，C 三个内存段形成一个大的空闲内存段 ABC，然后释放合并形成的 ABC 空闲内存段。

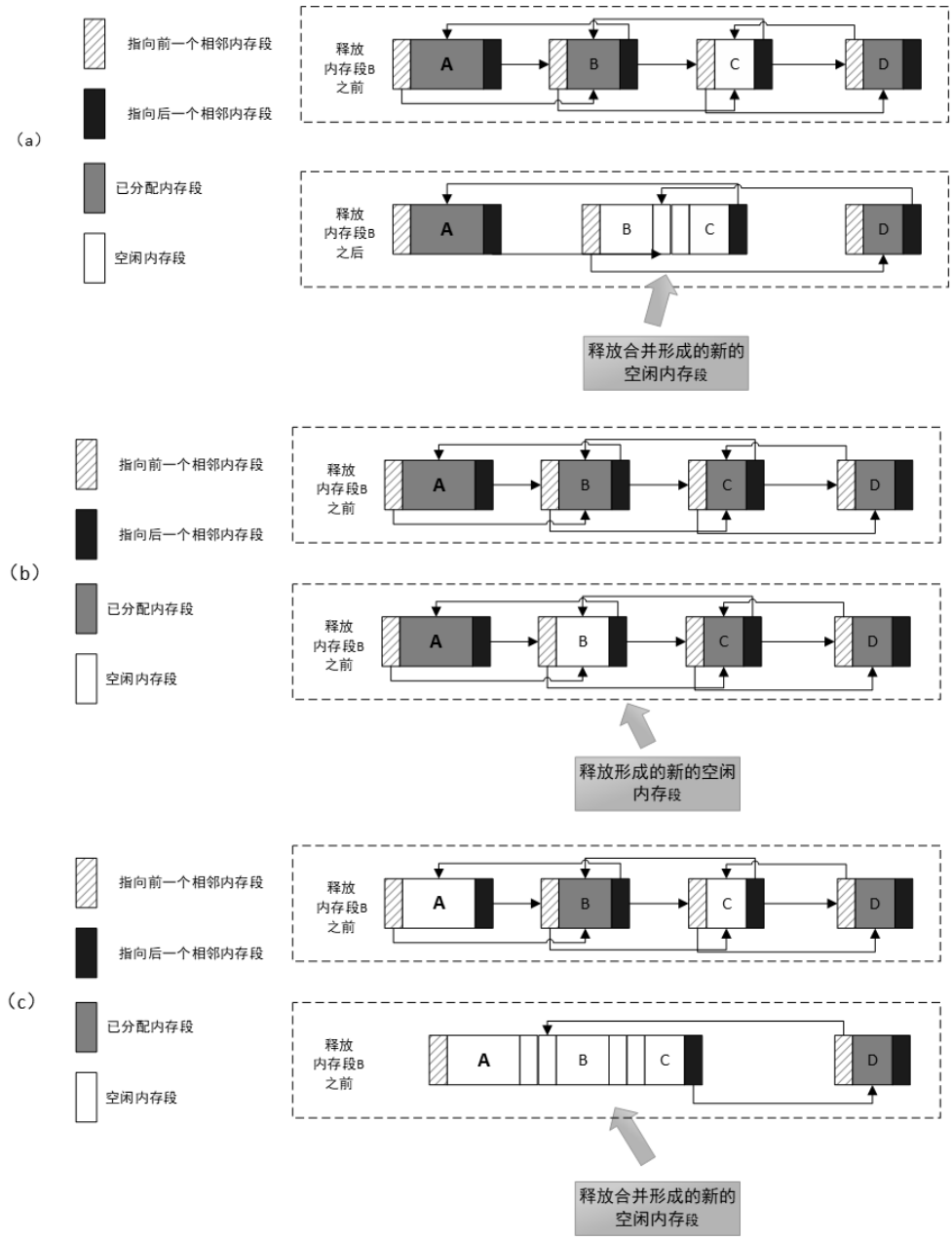


图 3-9-3 动态内存释放过程

3.2.3 内存管理接口

XiZi 内核中定义了一系列内存管理函数接口，包含内存初始化、内存申请、内存释放、内存信息统计等，如下所示：

表 3-2-1

函数原型	**void InitBoardMemory(void *start_phy_address, void *end_phy_address)**
函数功能	初始化静态和动态内存
参数说明	参数 start_phy_address 表示内存开始的物理地址；end_phy_address 表示内存结束的物理地址

返回值	无

表 3-2-2

函数原型	void *x_malloc(x_size_t size)
函数功能	分配一块合适大小的内存
参数说明	参数 size 表示分配内存的大小
返回值	若分配成功，则返回分配内存的首地址；否则返回 NONE

表 3-2-3

函数原型	void *x_realloc(void *pointer, x_size_t size)
函数功能	重新分配一块内存，将原内存块中的数据保存到新的内存中去，并将原内存块释放
参数说明	参数 pointer 表示原内存块的指针；size 表示需要重新分配的内存大小
返回值	若分配成功，则返回分配内存的首地址；否则返回 NONE

表 3-2-4

函数原型	void *x_calloc(xs_size_t count, x_size_t size)
函数功能	分配多个内存块并将内存块中的数据初始化为 0
参数说明	参数 count 表示需要分配的内存块数量；size 表示单个内存块的大小
返回值	若分配成功，则返回分配内存的首地址；否则返回 NONE

表 3-2-5

函数原型	void x_free(void *pointer)
函数功能	释放内存
参数说明	参数 pointer 表示指向需要被释放的内存
返回值	无

表 3-2-6

函数原型	void MemoryInfo (uint32 *total_memory, uint32 *used_memory, uint32 *max_used_memory)
函数功能	获取内存的统计信息
参数说明	参数 total_memory 表示内存总量；used_memory 表示已使用的内存；max_used_memory 表示最大分配内存
返回值	无

表 3-2-7

函数原型	void ShowMemory(void)
函数功能	打印内存信息 shell 命令
参数说明	无
返回值	无

3.3 中断管理

中断是指计算机运行过程中，出现某些意外情况需要主机干预时，机器能自动停止正在运行的程序并转入处理新情况的程序，处理完毕后又返回原被暂停的程序继续运行。中断的触发一般是由外设或者 CPU 内部产生，由中断控制器进行处理再通知到 CPU 进行中断响应过程，其要点如下：

- 外设或者 CPU 内部产生的这个中断来源称为中断源；
- CPU 响应中断转入到的新的处理程序称为中断服务程序；
- 中断处理完成后，处理器需要恢复之前的现场，简称“恢复现场”；
- 当多个中断源请求中断的时候，需要通过优先级区分中断，便于 CPU 进行中断的处理；
- 低优先级可以被高优先级打断，这个过程称之为中断嵌套；
- 中断可以被屏蔽；
- 所有的中断源都有一个编号，称为“中断号”；
- 每一个中断号通过中断向量表与中断服务程序一一对应，中断向量表保存的是所有的中断服务程序的入口地址，该入口地址被称之为中断向量。

XiZi 内核中，中断与体系结构密切相关，ARM 和 RISC-V 架构的中断响应各不相同。本节将从中断数据结构、中断管理机制和中断管理接口等部分进行介绍。

3.3.1 中断数据结构

XiZi 内核中断由 kernel/thread 目录内文件实现，主要包括线程 isr.c，体系结构由 arch 目录内文件实现。

I 中断管理结构体 InterruptServiceRoutines：

为适配不同体系结构，XiZi 内核中断由结构体 InterruptServiceRoutines 描述，其结构体定义如代码 3-3-1 所示。

代码 3-3-10 InterruptServiceRoutines 结构体

```
01  struct InterruptServiceRoutines
02  {
03      volatile uint16 isr_count;
04      volatile uint8  isr_switch_trigger_flag;
05      struct IrqDesc irq_table[ARCH_MAX_IRQ_NUM];
06      struct IsrDone *done;
07  };
08  extern struct InterruptServiceRoutines isrManager;
```

其中，isr_count 记录中断触发次数；isr_switch_trigger_flag 表示中断触发标志；irq_table 则表示系统注册中断向量表；done 表示中断操作回调函数。

II 中断向量表结构体 IrqDesc：

中断向量表由结构体 IrqDesc 描述，其结构体定义如代码 3-3-2 所示。

代码 3-3-2 IrqDesc 结构体

```
01  struct IrqDesc
02  {
03      IsrHandlerType handler;
04      void *param;
07  };
08  typedef void (*IsrHandlerType)(int vector, void *param);
```

其中，handler 表示该中断服务子函数；param 表示中断服务子函数传参。

III 中断操作回调函数结构体 IrqDone：

中断操作回调函数由结构体 IrqDone 描述，其结构体定义如代码 3-3-3 所示。

代码 3-3-3 IrqDone 结构体

```
01  struct IrqDone
02  {
03      x_bool (*isInIsr)();
04      int32   (*registerIrq)(uint32 irq_num, IsrHandlerType handler, void *arg);
05      int32   (*freeIrq)(uint32 irq_num);
06      int32   (*enableIrq)(uint32 irq_num);
07      int32   (*disableIrq)(uint32 irq_num);
08      void     (*handleIrq)(uint32 irq_num);
09      uint16 (*getCounter)() ;
10      void     (*incCounter)();
11      void     (*decCounter)();
12      uint8   (*getSwitchTrigerFlag)();
13     void     (*setSwitchTrigerFlag)();
14      void     (*clearSwitchTrigerFlag)();
15  };
```

其中，isInIsr 指向判断中断是否注册函数；registerIrq 指向中断注册函数；freeIrq 指向从中断管理中删除函数；enableIrq 指向中断使能函数；disableIrq 指向中断失效函数；handlerIrq 指向中断服务子函数；getCounter 指向获取中断触发次数函数；incCounter 指向中断触发次数增加函数；decCounter 指向中断触发次数减少函数；getSwitchTrigerFlag 指向获取中断触发标志函数；setSwitchTrigerFlag 指向设置中断触发标志函数；clearSwitchTrigerFlag 指向清除中断触发标志函数。

3.3.2 中断管理机制

I ARM Cortex-M 架构中断机制：

ARM Cortex-M 架构中断由中断控制器实现，其支持将中断事件请求作为中断输入到 NVIC 中断控制寄存器，唤醒 WFI 指令（Wait For Interrupt）；将中断事件请求作为事件输入，唤醒 WFE 指令（Wait For Event）。当一个中断触发时，NVIC 会检测该中断对应中断序号，和中断向量表中中断向量号匹配，MCU 跳转至该中断向量号对应中断服务程序执行，执行对应的操作。

中断向量表由 ARM 核中断处理向量和非 ARM 核中断处理向量组成，每个中断向量需要根据 MCU 要求设置在指定地址并依次排列（地址存入 NVIC 中断向量偏移寄存器），从而能够在中断触发时正确跳转相应的中断程序执行。以 edu-arm32（XiZi_IIoT/arch/cortex-m4/hc32f4a0/interrupt_vector.s）为例，ARM 核中断处理向量共有 16 个，从 0x0000_0000 到 0x0000_003C 依次排列，非 ARM 核中断处理向量共有 143 个，从 0x0000_0040 到 0x0000_027C 依次排列，如下表所示。

表 3-3-1 Cortex-M 中断向量表

地址	向量序号	号	中断源	说明	备注
ARM 核中断处理向量					
0x0000_0000	0	-	ARM core	Initial stack pointer	系统启动栈顶指针
0x0000_0004	1	-	ARM core	Initial PC	系统启动首个执行函数
0x0000_0008	2	-	ARM core	Non-maskable interrupt	不可屏蔽中断
0x0000_000C	3	-	ARM core	Hard Fault	硬件错误异常
0x0000_0010	4	-	ARM core	MemManage Fault	内存操作错误异常
0x0000_0014	5	-	ARM core	Bus Fault	指令/数据错误异常
0x0000_0018	6	-	ARM core	Usage Fault	操作错误异常
0x0000_001C	7	-	ARM core	Reserved	系统保留
0x0000_0020	8	-	ARM core	Reserved	系统保留
0x0000_0024	9	-	ARM core	Reserved	系统保留
0x0000_0028	10	-	ARM core	Reserved	系统保留
0x0000_002C	11	-	ARM core	Supervisor call（SVCall）	SVC 特权模式异常
0x0000_0030	12	-	ARM core	Debug Monitor	系统调试异常
0x0000_0034	13	-	ARM core	Reserved	系统保留
0x0000_0038	14	-	ARM core	PendableSrvReq	用于线程切换堆栈操作
0x0000_003C	15	-	ARM core	System tick timer	系统时钟中断
非 ARM 核中断处理向量					
0x0000_0040	16	0	INT_SEL0	寄存器 INTC_SEL0 选择的中断事件请求/软件中断	
……					
0x0000_023C	143	127	INT_SEL127	寄存器 INTC_SEL127 选择的中断事件请求/软件中断	
0x0000_0240	144	128	INT_VSSEL128	寄存器 INTC_VSSSEL128 选择的中断事件请求	
……					
0x0000_027C	159	143	INT_VSSEL143	寄存器 INTC_VSSSEL143 选择的中断事件请求	

II RISC-V 架构中断机制：

RISC-V 架构的中断和异常，从广义上来说，中断和异常属于一个概念。对于处理器而言，通常只区分为同步异常和异步异常：

- 同步异常，是指由于执行程序或者试图执行指令而产生的异常，例如，非法指令访问；
- 异步异常，最常见的异步异常是外部中断，例如外设触发一个外部中断。

RISC-V 架构定义的中断类型分为 4 种：

- 外部中断（External Interrupt），指来自处理器核外部的中断，例如 GPIO、UART 等产生的中断；
- 计时器中断（Timer Interrupt），指来自计时器的中断；
- 软件中断（Software Interrupt），指来自软件自己触发的中断；
- 调试中断（Debug Interrupt），专用于实现调试器（Debugger）。

RISC-V 中断控制器如图所示。其中，提供两种中断控制器，即 CLINT 控制器和 PLIC 控制器。

（1）CLINT 模块生成计时器中断和软件中断，CLINT 的全称为处理器核局部中断控制器（Core Local Interrupts Controller），主要用于产生计时器中断（Timer Interrupt）和软件中断（Software Interrupt）；

（2）PLIC 管理多个外部中断，PLIC 全称为平台级别中断控制器（Platform Level Interrupt Controller），它是 RISC-V 架构标准定义的系统中断控制器，主要用于多个外部中断源的优先级仲裁。

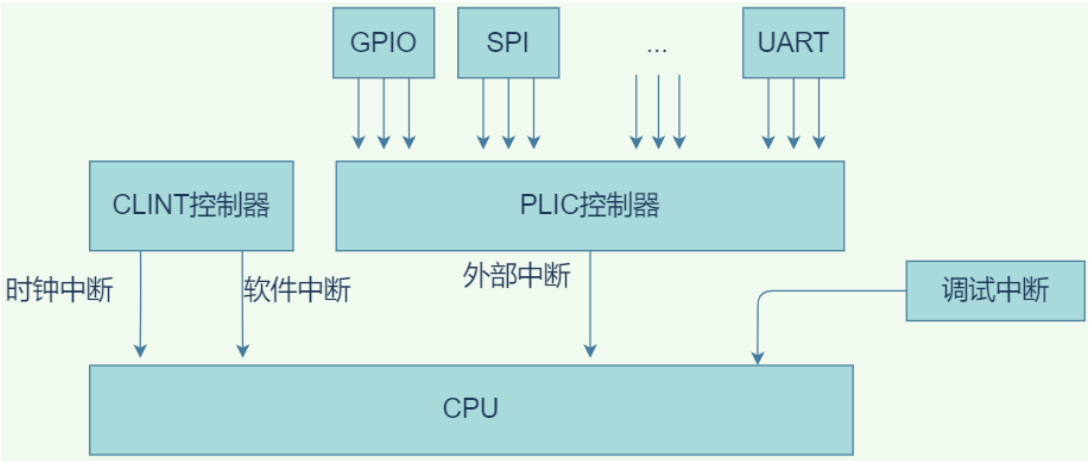


图 3-11-4 RISC-V 中断控制器

进入异常时，RISC-V 架构规定(以机器模式为例)：当前的程序执行流停止执行，直接跳转到 CSR 寄存器的 mtvec 定义的 PC 地址执行；硬件同时更新下列几个 CSR 寄存器。

- mcause (Machine Cause Register)：机器模式异常原因寄存器；
- mepc (Machine Exception Program Register)：机器模式异常 PC 寄存器；
- mtval (Machine Trap Value Register)：机器模式异常值寄存器；
- mstatus (Machine Status Register)：机器模式状态寄存器。

其中，mcause 寄存器的 Exception Code 域标识是何种异常或者何种中断，定义如表 3-3-2 所示：

表 3-12-2 Exception Code 域标识定义

Interrupt/Excep-tion mcause[XLEN-1]	Exception Code mcause[XLEN-2:0]	Description
1	1	Supervisor software interrupt
1	3	Machine software interrupt
1	5	Supervisor timer interrupt
1	7	Machine timer interrupt
1	9	Supervisor external interrupt
1	11	Machine external interrupt
0	0	Intruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	store address misaligned
0	7	Store access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	15	Store page fault

退出异常时，需要从异常服务程序中退出，并返回主程序。RISC-V 架构定义了一组专门的退出异常指令：MRET、SRET、URET，分别对应机器模式、监督模式、用户模式的退出。以 MRET 为例，当处理器执行 MRET 指令后，硬件行为如下：

（1）当前程序执行流程停止执行，跳转到 mepc 的地址运行；

（2）更新 mstatus 状态寄存器。

以上便完成了异常退出操作，具体实现可参考 XiZi_IIoT/arch/risc-v/k210/interrupt.c。

III XiZi 内核中断管理：

结合以上内容，可以知道 ARM 和 RISC-V 的中断机制存在差异，但从中断注册、中断向量表、中断服务程序到中断退出的整条中断响应链路上存在共性特征。因此，XiZi 内核根据中断工作原理，采用面向对象的方法，抽象中断管理操作，定义函数指针以适配不同体系架构中断数据结构和操作，即中断向量表结构体 IrqDesc 和中断操作回调函数结构体 IrqDone。

（1）中断注册：

XiZi 内核链接文件在.text 段定义了 SECTION(.isrtbl.idx)、SECTION(.isrtbl)段，通过 DECLARE_HW_IRQ 宏可实现按照中断名自动声明，内核初始化时调用函数 SysInitIsrManager 即可实现已声明中断的自动注册。

代码 3-3-4 XiZi_IIoT/board/edu-arm32/link.lds 文件

```
.text :
{
    KEEP(*(.isr_vector))
    ……
    . = ALIGN(4);
    __isrtbl_idx_start = .;
    KEEP(*(.isrtbl.idx))
    __isrtbl_start = .;
    KEEP(*(.isrtbl))
    __isrtbl_end = .;
    . = ALIGN(4);
    ……
}
```

其中，.isr_vector 段表示为 ARM Cortex-M 架构中断向量表 InterruptVectors 内存空间，需要注意的是 RISC-V 体系架构不需要定义；__isrtbl_idx_start 表示.isrtbl.idx 段起始地址；.isrtbl.idx 段表示中断向量表序号内存空间；__isrtbl_start 表示.isrtbl.idx 段结束地址，也表示.isrtbl 段起始地址，从内存排列上看，.isrtbl.idx 段和.isrtbl 段是地址连续的；.isrtbl 段表示中断向量表 IrqDesc 结构体内存空间；__isrtbl_end 表示.isrtbl 段结束地址。

代码 3-3-5 XiZi_IIoT/kernel/include/xs_isr.h 文件

```
……
#define DECLARE_HW_IRQ(_irq_num, _handler, _arg) \
    const uint32 __irq_desc_idx_##_handler SECTION(".isrtbl.idx") = _irq_num +
    ARCH_IRQ_NUM_OFFSET   ; \
    const struct IrqDesc __irq_desc_##_handler SECTION(".isrtbl") = { \
        .handler = _handler, \
        .param = _arg, \
    }
……
```

其中，##运算符可将__irq_desc_idx 和传参_handler 中断服务程序名字组合，形成新的 const 变量命名，其值为传参_irq_num 中断序号和 ARCH_IRQ_NUM_OFFSET（ARM 为 16，RISC-V 为 0）之和，并存入 SECTION(.isrtbl.idx)。同理，__irq_desc 和传参__handler 形成新的 const 结构体 struct IrqDesc，元素.handler 存入传参__handler，并存入 SECTION(.isrtbl)。

代码 3-3-6 XiZi_IIoT/kernel/thread/isr.c 文件

```
void SysInitIsrManager()
{
    extern int __isrtbl_idx_start;
    extern int __isrtbl_start;
    extern int __isrtbl_end;
    memset(&isrManager,0,sizeof(struct InterruptServiceRoutines));
    isrManager.done = &isrDone;
```

```
08
09      uint32 *index = (uint32 *)&__isrtbl_idx_start;
10      struct IrqDesc *desc = (struct IrqDesc *)&__isrtbl_start;
11
12      while (desc != (struct IrqDesc *)&__isrtbl_end)
13          isrManager.irq_table[*index++] = *desc++;
14  }
```

其中，XiZi 内核启动中调用 SysInitIsrManager 函数，从定义的.isrtbl.idx 段和.isrtbl 段中获取所注册的中断结构体 IrqDesc，并存入 isrManager.irq_table 中，纳入中断管理数据结构，从而实现对中断的抽象。

需要注意的是，芯片厂商 SDK 往往会提供中断注册 SDK 接口，与 XiZi 内核提供的中断抽象接口不冲突，用户在开发中可根据实际情况选择合适的接口实现中断注册。

代码 3-3-7 XiZi_IIoT/board/edu-arm32/third_party_driver/common/hardware_irq.c 文件

```
01  x_err_t hc32_install_irq_handler(struct Hc32IrqConfig *irq_config,
02                                   void (*irq_hdr)(void),
03                                   x_bool irq_enable)
04  {
05      x_err_t result = -ERROR;
06      stc_irq_signin_config_t stcIrqSignConfig;
07
08      NULL_PARAM_CHECK(irq_config);
09      NULL_PARAM_CHECK(irq_hdr);
10
11      stcIrqSignConfig.enIRQn = irq_config->irq_num;
12      stcIrqSignConfig.enIntSrc = irq_config->int_src;
13      stcIrqSignConfig.pfnCallback = irq_hdr;
14      if (LL_OK == INTC_IrqSignIn(&stcIrqSignConfig)) {
15          NVIC_ClearPendingIRQ(stcIrqSignConfig.enIRQn);
16          NVIC_SetPriority(stcIrqSignConfig.enIRQn, irq_config->irq_prio);
17          if (irq_enable) {
18              NVIC_EnableIRQ(stcIrqSignConfig.enIRQn);
19          } else {
20              NVIC_DisableIRQ(stcIrqSignConfig.enIRQn);
21          }
22          result = EOK;
23      }
24      return result;
25  }
```

其中，edu-arm32 使用 hc32_install_irq_handler 函数注册 NVIC 中断。

（2）中断服务程序：

在中断触发后，CPU 将当前运行时刻的上下文寄存器保存到中断栈中，即保存上下文

信息，关闭中断响应，然后执行相应的中断服务程序，完成后进行线程切换，最后重新打开中断响应，CPU 再将之前保存的上下文信息恢复，重新执行被打断的指令。XiZi 内核中，中断服务程序根据中断类型编写，例如 UART 中断服务程序则完成 UART 相关操作、时钟中断服务程序则完成时钟计数相关操作，遵循的原则是中断服务程序尽可能快速执行结束，避免因为关闭中断响应过久而影响其他中断的响应，从而影响系统的实时性。

XiZi 内核中支持两种形式的中断服务程序，一种是根据中断向量表中定义函数命名实现，如 SysTick_Handler，另外一种是使用 XiZi 内核中断抽象函数 IsrEntry 实现，如以下代码所示。

代码 3-3-8 XiZi_IIoT/arch/arm/cortex-m4/hc32f4a0/interrupt_vector.S 文件

```
01  InterruptVectors:
02                  .long       __StackTop              /*      Top of Stack */
03                  .long       Reset_Handler           /*      Reset Handler */
04                  .long       NMI_Handler             /* -14 NMI Handler */
05                  .long       HardFaultHandler        /* -13 Hard Fault Handler */
06                  .long       BusFault_Handler        /* -11 Bus Fault Handler */
07                  .long       UsageFault_Handler      /* -10 Usage Fault Handler */
08                  .long       0                       /*      Reserved */
09                  .long       0                       /*      Reserved */
10                  .long       0                       /*      Reserved */
11                  .long       0                       /*      Reserved */
12                  .long       IsrEntry                /*   -5 SVCall Handler */
13                  .long       IsrEntry                /*   -4 Debug Monitor Handler */
14                  .long       0                       /*      Reserved */
15                  .long       PendSV_Handler          /*   -2 PendSV Handler */
16                  .long       SysTick_Handler         /*   -1 SysTick Handler */
17
18                  /* Interrupts */
19                  .long       IsrEntry
20                  .long       IsrEntry
21                  ……
22                  .long       IRQ030_Handler
23                  .long       IsrEntry
24                  ……
25                  .long       IsrEntry
```

代码 3-3-9 XiZi_IIoT/board/edu-arm32/board.c 文件

```
01  void SysTick_Handler(void)
02  {
03      x_base lock = 0;
04      lock = DISABLE_INTERRUPT();
05
06      TickAndTaskTimesliceUpdate();
07
```

```
08          ENABLE_INTERRUPT(lock);
09      }
```

代码 3-3-10 XiZi_IIoT/arch/arm/cortex-m4/hc32f4a0/interrupt.c 文件

```
01  void IsrEntry()
02  {
03          uint32 ipsr;
04
05          __asm__ volatile("MRS %0, IPSR" : "=r"(ipsr));
06
07          isrManager.done->incCounter();
08          isrManager.done->handleIrq(ipsr);
09          KTaskOsAssignAfterIrq(NONE);
10          isrManager.done->decCounter();
11  }
```

3.3.3 中断管理接口

XiZi 内核中定义了一系列中断管理函数接口，包含中断管理初始化、中断注册、中断释放、中断使能、中断关闭、中断触发统计等，如下所示：

表 3-3-4

函数原型	**SysInitIsrManager()**
函数功能	初始化中断管理结构体和中断 irq_table，填充定义 isrtbl 段
参数说明	无
返回值	无

表 3-3-5

函数原型	**int32 RegisterHwIrq(uint32 irq_num, IsrHandlerType handler, void *arg)**
函数功能	注册一个中断
参数说明	参数 irq_num 表示中断在对应架构中的序号；参数 handler 表示该中断对应中断服务程序入口；参数 arg 表示中断服务程序传参
返回值	若注册成功，返回 EOK；否则返回-ERROR

表 3-3-6

函数原型	**int32 FreeHwIrq(uint32 irq_num)**
函数功能	释放一个中断
参数说明	参数 irq_num 表示中断在对应架构中的序号
返回值	若释放成功，返回 EOK；否则返回-ERROR

表 3-3-7

函数原型	**int32 EnableHwIrq(uint32 irq_num)**

函数功能	使能一个中断
参数说明	参数 irq_num 表示中断在对应架构中的序号
返回值	若成功，返回 EOK；否则返回-ERROR

表 3-3-8

函数原型	int32 DisableHwIrq(uint32 irq_num)
函数功能	关闭一个中断
参数说明	参数 irq_num 表示中断在对应架构中的序号
返回值	若成功，返回 EOK；否则返回-ERROR

表 3-3-9

函数原型	uint16 GetIsrCounter()
函数功能	获取系统触发中断个数
参数说明	无
返回值	返回系统触发中断个数

3.4. 时钟管理

每一个操作系统都需要一个时钟计数，提供“心跳”计数 TICK，该时钟可以提供系统处理所有和时钟相关的事件，例如线程时间片、信号量等内核对象超时统计等。XiZi 内核通过芯片提供的硬件定时器产生的周期性中断 SysTick_Handler 进行计数，同时为解决系统硬件定时器个数有限的问题，实现了一套软件定时器，为内核对象提供精确的定时服务和功能。

3.4.1 时钟数据结构

XiZi 内核时钟硬件定时器由 kernel/thread/tick.c 文件实现，头文件为 xs_ktick.h，软件定时器由 kernel/thread/softtimer.c 文件实现，头文件为 xs_timer.h。

I 硬件定时器数据结构：

硬件定时器数据结构由一个 32 位无符号数组成，每次定时器中断触发时该值加一，作为系统运行的时钟统计。

代码 3-4-13 x_ticks_t 数据结构

```
01    typedef uint32 x_ticks_t;
```

II 软件定时器 Timer 结构体：

软件定时器模块根据硬件定时器 TICK 值提供软件定时服务，用户可以设定固定的超时时间，当系统运行的节拍数到达用户设定的超时时间时，便执行用户定义的超时回调函数进行业务处理。

代码 3-4-2 Timer 结构体

```
01  struct Timer
02  {
03      struct IdNode      id_node;
04      char                name[NAME_NUM_MAX];
05      uint8               active_status;
06      uint8               trigger_mode;
07      void (*func_callback)(void *param);
08      void                *param;
09      x_ticks_t           origin_timeslice;
10      x_ticks_t           deadline_timeslice;
11      DoubleLinklistType  link;
12      DoubleLinklistType  sortlist;
13      uint8 prio;
14      struct Work * t_work;
15      struct TimerDone* done;
16  }
17  typedef struct Timer *TimerType;
```

其中，参数 id_node 表示软件定时器 ID，标识唯一的软件定时器；参数 name 表示软件定时器名称；参数 active_status 表示定时器的激活状态；参数 trigger_mode 表示定时器的触发模式；参数 func_callback 表示定时器超时回调函数；参数 param 表示定时器超时回调函数的参数；参数 origin_timeslice 表示初始化的超时时间；参数 deadline_timeslice 表示截止时间；参数 link 表示系统中所有定时器构成的链表；参数 sortlist 表示处于激活态的定时器链表，并按时间排序；参数 prio 表示记录使用定时器处理回调的线程优先级；参数 t_work 表示定时器所用到的工作队列；参数 done 表示定时器的函数操作集。

3.4.2 时钟管理机制

XiZi 内核中定义了一个变量 TICK_PER_SECOND，该变量解释为每秒钟支持的节拍数（TICK）。当 TICK_PER_SECOND 配置为 100，则 TICK 节拍表示 10 毫秒，即一个时间片（timeslice）为 10 毫秒；当 TICK_PER_SECOND 配置为 1000，则 TICK 节拍表示 1 毫秒，即一个时间片（timeslice）为 1 毫秒。在节拍配置成功后，系统支持的调度算法、软件定时器等依赖于该时钟周期服务的事件就可以正常工作，例如在支持的时间片轮转算法当中，每进行的一次线程切换为 10 个 TICK。

在每一个定时器中断到来时，系统进行常规的中断执行流，定时器按照每一个 TICK 单位进行触发中断，在中断服务程序当中进行自增，并对当前运行线程的时间片进行计算，同步可处理定时器的计数时间，如下图所示：

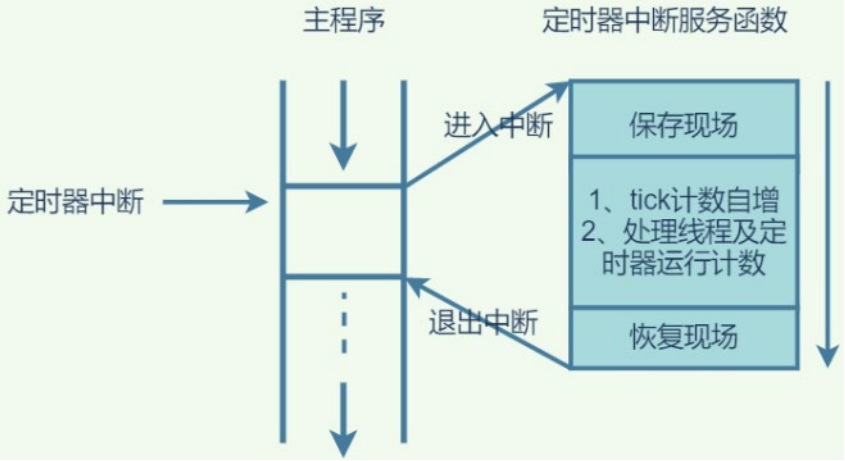


图 3-14-1 TICK 计数的详细执行流程

3.4.3 时钟管理接口

XiZi 内核中定义了一系列时钟管理函数接口，包含 TICK 配置、软件定时器配置、软件定时器开启等，如下所示：

表 3-4-1

函数原型	x_ticks_t CalculteTickFromTimeMs(uint32 ms)
函数功能	输入毫秒时间，并转化成节拍数输出
参数说明	参数 ms 表示毫秒时间
返回值	TICK 值

表 3-4-2

函数原型	uint32 CalculteTimeMsFromTick(x_ticks_t ticks)
函数功能	输入节拍，并转化成毫秒时间输出
参数说明	参数 ticks 表示 TICK 值
返回值	毫秒值

表 3-4-3

函数原型	x_ticks_t CurrentTicksGain(void)
函数功能	获取当前的时钟节拍数
参数说明	无
返回值	TICK 值

表 3-4-4

函数原型	int32 KCreateTimer(const char *name, void (*timeout)(void *parameter), void *parameter, x_ticks_t time, uint8 trigger_mode)
函数功能	创建一个内核软件定时器，并返回创建成功的软件定时器的 ID，ID 默认范围 0-255，可配置

参数说明	参数 name 表示定时器名称；timeout 表示软件定时器回调函数；parameter 表示软件定时器回调函数参数；time 表示配置需要等待的超时时间，单位 tick；trigger_mod 表示触发方式，可以配置为宏 TRIGGE_WAY_ONCE 和 TRIGGE_WAY_PERIODIC
返回值	若创建成功返回软件定时器的 ID，若创建失败则返回 NONE

表 3-4-5

函数原型	**x_err_t KDeleteTimer(int32 timer_id)**
函数功能	删除一个软件定时器
参数说明	参数 timer_id 表示待删除的软件定时器 ID
返回值	返回 EOK

表 3-4-6

函数原型	**x_err_t KTimerStartRun(int32 timer_id)**
函数功能	启动一个软件定时器
参数说明	参数 timer_id 表示已创建且待运行的软件定时器 ID
返回值	返回 EOK

表 3-4-7

函数原型	**x_err_t KTimerQuitRun(int32 timer_id)**
函数功能	停止一个软件定时器
参数说明	参数 timer_id 表示待停止运行的软件定时器 ID
返回值	若停止成功，返回 EOK，若失败则返回-ERROR

表 3-4-8

函数原型	**x_err_t KTimerModify(int32 timer_id, x_tick_t ticks)**
函数功能	修改一个软件定时器的超时时间
参数说明	参数 timer_id 表示待修改超时时间的软件定时器 ID；ticks 表示超时时间
返回值	若修改成功，返回 EOK，若失败则返回-EINVALED

3.5 文件系统

嵌入式操作系统面向海量物联网节点设备，需要在存储介质上支持大量数据的存储、解析和分发，要求操作系统能够高效地处理，文件系统便是针对这种问题而设计。文件系统本质上是一种实现数据存储、组织、访问和获取的抽象机制，其基本存储单位是文件，所有数据按文件形式存储、索引。目前操作系统领域的文件系统种类很多，分别面向不同的应用场景，如 FATFS、Jffs2、LWext4、NFS 等，为适配不同文件系统的特性，XiZi 内核支持虚拟文件系统 IoT-VFS，为用户提供树形的文件组织结构，对上提供统一的 VFS 虚拟文件系统接口，对下兼容不同文件系统，目前 XiZi 内核存储设备支持 FATFS，外设设备支持 Device-VFS，即将各类外设设备抽象为文件，通过/dev 路径的操作可实现对设备的读写。

3.5.1 文件系统数据结构

XiZi 内核虚拟文件系统由 kernel/fs/shared 目录内文件实现，移植 FATFS 文件系统由 kernel/fs/fatfs 目录内文件实现，设备文件系统 Device-VFS 由 kernel/fs/iot-devicefile 目录内文件实现。

I 文件句柄结构体 FileDescriptor：

文件系统由文件句柄结构体描述，如下所示。

代码 3-5-15 FileDescriptor 结构体

```
01  struct FileDescriptor
02  {
03      char *path;
04      enum x_FileType type;
05      void *data;
06      struct MountPoint *mntp;
07      off_t pos;
08  }
```

其中，参数 path 表示文件路径；参数 type 表示文件类型，由枚举 x_Filetype 定义；参数 data 表示文件系统数据指针；参数 mntp 表示文件系统挂载节点结构体指针；参数 pos 表示文件读写时偏移值。

Ⅱ 文件系统挂载结构体 MountPoint：

文件系统挂载节点由结构体 MountPoint 描述，支持将文件系统挂载到特定存储设备上，是操作系统能执行文件系统命令，如 ls、mkdir 等，的前提条件。MountPoint 结构体如下所示。

代码 3-6-2 MountPoint 结构体

```
01  struct MountPoint
02  {
03      struct SysDoubleLinklistNode node;
04      enum FilesystemType fs_type;
05      char *mnt_point;
06      size_t mnt_point_len;
07      void *fs_data;
08      BusType bus;
09      HardwareDevType dev;
10      DriverType drv;
11      struct Filesystem *fs;
12  }
```

其中，参数 node 表示挂载节点，系统采用双向链表管理所有挂载节点；参数 fs_type 表示所挂载的文件系统类型，由枚举 FilesystemType 定义；参数 mnt_point 表示挂载文件目录字符串指针，如“\”根目录；参数 mnt_point_len 表示挂载文件目录字符串长度；参数 fs_data 表示挂载文件系统数据指针；参数 bus 表示挂载设备文件系统设备总线结构体；参数 dev 表示挂载设备文件系统设备结构体；参数 drv 表示挂载设备文件系统设备驱动结构体；参数 fs 表示文件系统操作回调函数。

III 文件系统操作回调结构体 Filesystem：

文件系统操作回调函数由结构体 Filesystem 描述，支持文件系统操作函数，如 open、close、read、write、ioctl、opendir、closedir 等，如下所示。

代码 3-6-3 Filesystem 结构体

```
01  struct Filesystem
02  {
03      int (*open)(struct FileDescriptor *fdp, const char *path);
04      int (*close)(struct FileDescriptor *fdp);
05      ssize_t (*read)(struct FileDescriptor *fdp, void *dst, size_t len);
06      ssize_t (*write)(struct FileDescriptor *fdp, const void *src, size_t len);
07      int (*seek)(struct FileDescriptor *fdp, off_t offset, int whence, off_t *new_offset);
08      off_t (*tell)(struct FileDescriptor *fdp);
09      int (*truncate)(struct FileDescriptor *fdp, off_t length);
10      int (*sync)(struct FileDescriptor *fdp);
11
12      int (*ioctl)(struct FileDescriptor *fdp, int cmd, void *args);
13      int (*poll)(struct FileDescriptor *fdp, struct Pollreq *req);
14
15      int (*opendir)(struct FileDescriptor *fdp, const char *path);
16      int (*closedir)(struct FileDescriptor *fdp);
17      int (*readdir)(struct FileDescriptor *fdp, struct dirent *dirent);
18      int (*seekdir)(struct FileDescriptor *fdp, off_t offset);
19
20      int (*mount)(struct MountPoint *mp);
21      int (*unmount)(struct MountPoint *mp);
22      int (*unlink)(struct MountPoint *mp, const char *path);
23      int (*rename)(struct MountPoint *mp, const char *from, const char *to);
24      int (*mkdir)(struct MountPoint *mp, const char *path);
25      int (*stat)(struct MountPoint *mp, const char *path, struct FileStat *buf);
26      int (*statvfs)(struct MountPoint *mp, const char *path, struct statfs *buf);
27  }
```

其中，参数 open 表示文件系统打开操作；参数 close 表示文件系统关闭操作；参数 read 表示文件系统读操作；参数 write 表示文件系统写操作；参数 seek 表示文件系统定位指定位置操作；参数 tell 表示文件系统获取当前读写位置操作；参数 truncate 表示文件系统设置文件为指定长度操作；参数 sync 表示文件系统将数据写入存储介质操作；参数 ioctl 表示文件

系统配置操作；参数 poll 表示文件系统等待文件句柄就绪操作；参数 opendir 表示文件系统打开特定目录操作；参数 closedir 表示文件系统关闭特定目录操作；参数 readdir 表示文件系统读特定目录操作；参数 seekdir 表示定位文件系统操作；参数 mount 表示挂载文件系统操作；参数 unmount 表示取消挂载文件系统操作；参数 unlink 表示删除指定目录项操作；参数 rename 表示重命名目录操作；参数 mkdir 表示创建目录操作；参数 stat 表示获取文件系统状态信息操作；参数 statvfs 表示获取虚拟文件系统状态信息操作。

3.5.2 虚拟文件系统

XiZi 内核所设计的虚拟文件系统 IoT-VFS 构件，为用户提供了树形的文件组织结构。当前版本的 IoT-VFS 在功能上保证了 FATFS 的 VFS 特性，同时允许用户使用统一的 Device-VFS 的接口来操作设备。IoT-VFS 具有以下特性：

- 统一采用 UNIX 风格路径格式，目录与文件层级清晰分明；
- 面向用户提供 POSIX 文件接口，降低应用程序移植难度；
- 可以在任意路径挂载不同文件系统，并且可以便捷地添加对新的文件系统的支持。

在 IoT-VFS 的支持下，应用程序开发者无须了解底层文件系统的接口细节，只需使用标准 POSIX 文件接口进行应用开发，对于已有的 POSIX 应用程序，也可以直接进行源代码级别的移植。使用 IoT-VFS 的应用程序架构如图 3-5-1 所示。

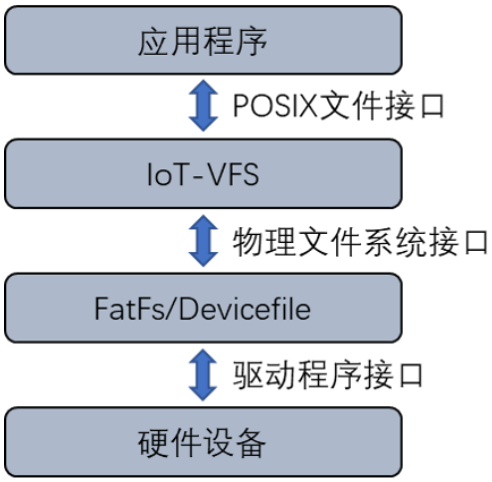


图 3-16-5 IoT-VFS 的应用程序架构

虚拟文件系统 IoT-VFS 支持 FATFS，首先需要定义基于文件系统操作回调结构体 Filesystem，实现 open、close、read、write 等回调函数，最终注册到 IoT-VFS 中，待挂载至相应存储设备，如以下代码所示。

代码 3-6-4 XiZi_IIoT/fs/fatfs/fatfs.c 文件

```
01  /* fatfs filesystem interface */
02  static struct Filesystem fatfs_fs = {
03          .open = FatfsOpen,
04          .close = FatfsClose,
05          .read = FatfsRead,
06          .write = FatfsWrite,
```

```
07          .seek = FatfsSeek,
08          .tell = FatfsTell,
09          .truncate = FatfsTruncate,
10          .sync = FatfsSync,
11          .ioctl = NULL,
12          .poll = NULL,
13          .opendir = FatfsOpendir,
14          .closedir = FatfsClosedir,
15          .readdir = FatfsReaddir,
16          .seekdir = NULL,
17          .mount = FatfsMount,
18          .unmount = FatfsUnmount,
19          .unlink = FatfsUnlink,
20          .rename = FatfsRename,
21          .mkdir = FatfsMkdir,
22          .stat = FatfsStat,
23          .statvfs = FatfsStatvfs,
24  };
25
26  int FatfsInit()
27  {
28          return RegisterFilesystem(FSTYPE_FATFS, &fatfs_fs);
29  }
```

代码 3-6-5 XiZi_IIoT/board/edu-arm32/third_party_driver/sdio/connect_sdio.c 文件

```
01  #if defined(FS_VFS) && defined(MOUNT_SDCARD_FS)
02  #include <iot-vfs.h>
03  static int MountSDCardFs(enum FilesystemType fs_type)
04  {
05          if (MountFilesystem(SDIO_BUS_NAME, SDIO_DEVICE_NAME,
SDIO_DRIVER_NAME, fs_type, "/") == 0)
06                  KPrintf("Sd card mount to '/'");
07          else
08                  KPrintf("Sd card mount to '/' failed!");
09          return 0;
10  }
11  #endif
26  int FatfsInit()
27  {
28          return RegisterFilesystem(FSTYPE_FATFS, &fatfs_fs);
29  }
```

同理，虚拟文件系统 IoT-VFS 支持 Device-VFS，首先需要定义基于文件系统操作回调结构体 Filesystem，实现部分回调函数，如 open、close、read、write、ioctl、mount，最终注册到 IoT-VFS 中，挂载到路径“/dev”，应用层便可操作该路径下的外设驱动，如以下代码

所示

代码 3-6-6 XiZi_IIoT/fs/iot-devicefile/iot-devicefile.c 文件

```
01  static struct Filesystem devicefile_fs = {
02          .open = DevicefileOpen,
03          .close = DevicefileClose,
04          .read = DevicefileRead,
05          .write = DevicefileWrite,
06          .ioctl = DevicefileIoctl,
07          .mount = DevicefileMount,
08  };
09
10  int DevicefileInit()
11  {
12          RegisterFilesystem(FSTYPE_IOTDEVICEFILE, &devicefile_fs);
13
14          return 0;
15  }
```

代码 3-6-7 XiZi_IIoT/fs/shared/src/iot-vfs.c 文件

```
01  int VfsInit()
02  {
03          InitDoubleLinkList(&mnt_list);
04
05          mnt_list_lock = KMutexCreate();
06          working_dir_lock = KMutexCreate();
07          fdtable_lock = KMutexCreate();
08
09          strcpy(working_dir, "/");
10
11          extern int DevicefileInit();
26          DevicefileInit();
27          MountFilesystem(NULL, NULL, NULL, FSTYPE_IOTDEVICEFILE, "/dev");
28
29          return 0;
30  }
```

3.5.3 文件系统接口

XiZi 内核中定义了一系列文件系统函数接口，包含虚拟文件系统初始化、文件系统注册、挂载等，如下所示：

表 3-5-1

函数原型	int VfsInit()
函数功能	虚拟文件系统初始化，系统启动阶段调用
参数说明	无
返回值	返回 EOK

表 3-5-2

函数原型	int RegisterFilesystem(enum FilesystemType fs_type, struct Filesystem *fs)
函数功能	文件系统注册到虚拟文件系统 IoT-VFS 中
参数说明	参数 fs_type 表示文件系统类型；参数 fs 表示文件系统描述符指针
返回值	若注册成功，返回 EOK；若失败，则返回-EINVAL

表 3-5-3

函数原型	int MountFilesystem(const char *bus_name, const char *dev_name, const char *drv_name, enum FilesystemType fs_type, const char *path)
函数功能	将文件系统挂载到指定存储设备，并指定根目录
参数说明	参数 bus_name 表示存储设备总线名称；参数 dev_name 表示存储设备名称；参数 drv_name 表示存储设备驱动名称；参数 fs_type 表示挂载文件系统类型；参数 path 表示挂载文件系统根目录
返回值	若挂载成功，返回 EOK；若失败，存储设备异常返回-ENXIO，不支持的文件系统返回-EINVAL，挂载节点内存申请失败则返回-ENOMEM

表 3-5-4

函数原型	int UnmountFileSystem(const char *path)
函数功能	将文件系统从指定根目录取消挂载
参数说明	参数 path 表示要取消挂载文件系统根目录
返回值	若取消挂载成功，返回 EOK；若失败，返回-EINVAL

3.6 驱动管理

多数嵌入式操作系统对驱动的抽象采用以物理外设实体为中心的方式，采用驱动(DRIVER)和设备(DEVICE)一一对应的模式开发驱动，每新增一个外设都需要相应地开发驱动 DRIVER 和设备 DEVICE 逻辑，涉及底层寄存器和应用层业务，开发难度和工作量相对较大，同时操作系统支持外设的数量和种类越多，工程中的驱动实现代码越多，部分同类驱动的代码存在冗余情况，不利于裁剪优化。

为解决以上问题，XiZi 内核采用 BUS 总线驱动框架，利用一套 BUS 总线管理驱动 DRIVER 和设备 DEVICE，即同种设备总线管理下，多个设备 DEVICE 复用一个驱动 DRIVER，数据结构上以 BUS 为基类，分发多种子类对应多种外设，统一管理驱动外设。以 SPI_BUS 总线为例，SPI_BUS 总线创建后，首先会注册 SPI_DRIVER 驱动相关实现，之后系统接入的所有 SPI_DEVICE 都会注册挂载到 SPI_BUS 总线上，DRIVER 驱动和 DEVICE 设备通过 bus_name 绑定上，实现设备和驱动相互对应的配置，这样即可实现用一套 BUS 数据结构管理其挂载的 DEVICE 设备和 DRIVER 驱动的目的。对于上层应用来说，通过获取 SPI_BUS

数据结构即可知道当前系统中挂载的 SPI_DEVICE 个数，利用 BUS 回调函数便可满足配置、数据交互需求，而不用关心具体的硬件细节，且底层新增或删减外设的操作由框架层统一管理，上层应用模块依旧可以复用，这套框架层次化管理界限清晰，驱动和应用解耦、复用功能凸显，应用层开发友好。

3.6.1 驱动模型数据结构

XiZi 内核驱动模型由 kernel/resources/bus.c 文件实现，头文件为 bus.h。

I 设备总线 Bus 结构体：

设备总线 Bus 结构体定义了外设设备总线的基本信息和回调函数，采用双向链表管理系统中所有的设备总线节点，Bus 结构体如以下代码所示。

代码 3-6-17 Bus 结构体

```
01  struct Bus
02  {
03      int8 bus_name[NAME_NUM_MAX];
04      enum BusType bus_type;
05      enum BusState bus_state;
06
07      int32 (*match)(struct Driver *driver, struct HardwareDev *device);
08      int bus_lock;
09
10      struct HardwareDev *owner_haldev;
11      struct Driver *owner_driver;
12
13      void *private_data;
14
15      /*manage the drv of the bus*/
16      uint8 driver_cnt;
17      uint8 bus_drvlink_flag;
18      DoubleLinklistType bus_drvlink;
19
20      /*manage the dev of the bus*/
21      uint8 haldev_cnt;
22      uint8 bus_devlink_flag;
23      DoubleLinklistType bus_devlink;
24
25      uint8 bus_cnt;
26      uint8 bus_link_flag;
27      DoubleLinklistType    bus_link;
28  }
```

其中，参数 bus_name 表示设备总线的名字；参数 bus_type 表示设备总线的类型，由枚举 BusType 定义；参数 bus_state 表示设备总线的状态，由枚举 BusState 定义；参数 match 表示用于设备和设备驱动匹配的回调函数；参数 bus_lock 表示设备总线互斥锁，应用于同步场景保护；参数 owner_haldev 表示挂载到该设备总线上的设备描述符；参数 owner_driver 表示挂载到该设备总线上的设备驱动描述符；参数 private_data 表示设备总线私有数据指针；参数 driver_cnt 表示挂载到该设备总线上的设备驱动个数；参数 bus_drvlink_flag 表示挂载到设备总线上的设备驱动双向链表初始化 flag；参数 bus_drvlink 表示挂载到设备总线上的设备驱动双向链表；参数 haldev_cnt 表示挂载到该设备总线上的设备个数；参数 bus_devlink_flag 表示挂载到设备总线上的设备双向链表初始化 flag；参数 bus_devlink 表示挂载到设备总线上的设备双向链表；参数 bus_cnt 表示设备总线个数；参数 bus_link_flag 表示设备总线双向链表初始化 flag；参数 bus_link 表示设备总线双向链表。

II 设备 HardwareDev 结构体：

设备 HardwareDev 结构体定义了外设设备基本信息和回调函数，采用双向链表管理系统中所有的设备节点，并挂载到对应外设驱动总线上，设备 HardwareDev 结构体如以下代码所示。

代码 3-6-2 HardwareDev 结构体

```
01  struct HardwareDev
02  {
03      int8 dev_name[NAME_NUM_MAX];
04      enum DevType dev_type;
05      enum DevState dev_state;
06
07      const struct HalDevDone *dev_done;
08
09      int (*dev_recv_callback) (void *dev, x_size_t length);
10      int (*dev_block_control) (struct HardwareDev *dev, struct HalDevBlockParam
*block_param);
11
12      struct Bus *owner_bus;
13      void *private_data;
14
15      int32 dev_sem;
16      DoubleLinklistType   dev_link;
17  }
```

其中，参数 dev_name 表示设备名字；参数 dev_type 表示设备类型，由枚举 DevType 定义；参数 dev_state 表示设备状态，由枚举 DevState 定义；参数 dev_done 表示设备操作回调函数；参数 dev_recv_callback 表示设备接收数据后执行回调函数；参数 dev_block_control 表示块设备配置回调函数；参数 owner_bus 表示设备所挂载的设备总线描述符；参数 private_data 表示设备私有数据指针；参数 dev_sem 表示设备操作信号量；参数 dev_link 表示设备管理双向链表。

III 设备驱动 Driver 结构体：

设备驱动 Driver 结构体定义了外设设备驱动基本信息和回调函数，采用双向链表管理系统中所有的设备驱动节点，并挂载到对应外设驱动总线上，设备驱动 Driver 结构体如以下代码所示。

代码 3-6-3 Driver 结构体

```
01  struct Driver
02  {
03      int8 drv_name[NAME_NUM_MAX];
04      enum DriverType driver_type;
05      enum DriverState driver_state;
06
07      uint32 (*configure)(void *drv, struct BusConfigureInfo *configure_info);
08
09      struct Bus *owner_bus;
10      void *private_data;
11
12      DoubleLinklistType   drv_link;
13  }
```

其中，参数 drv_name 表示设备驱动名字；参数 drv_type 表示设备驱动类型，由枚举 DrvType 定义；参数 drv_state 表示设备驱动状态，由枚举 DrvState 定义；参数 configure 表示设备驱动配置回调函数；参数 owner_bus 表示设备驱动所挂载的设备总线描述符；参数 private_data 表示设备驱动私有数据指针；参数 drv_link 表示设备驱动管理双向链表。

3.6.2 驱动模型

XiUOS 的驱动框架 resources/xxx/文件目录结构如下表所示：

表 3-6-18 驱动框架 resources/xxx/文件目录结构

文件名	描述
bus.c	bus 驱动框架实现，提供给上层应用调用
bus_xxx.c	xxx_bus 子类实现，如 spi_bus、i2c_bus、serial_bus 等
dev_xxx.c	xxx_dev 子类实现，如 spi_dev、i2c_dev、serial_dev 等
drv_xxx.c	xxx_drv 子类实现，如 spi_drv、i2c_drv、serial_drv 等

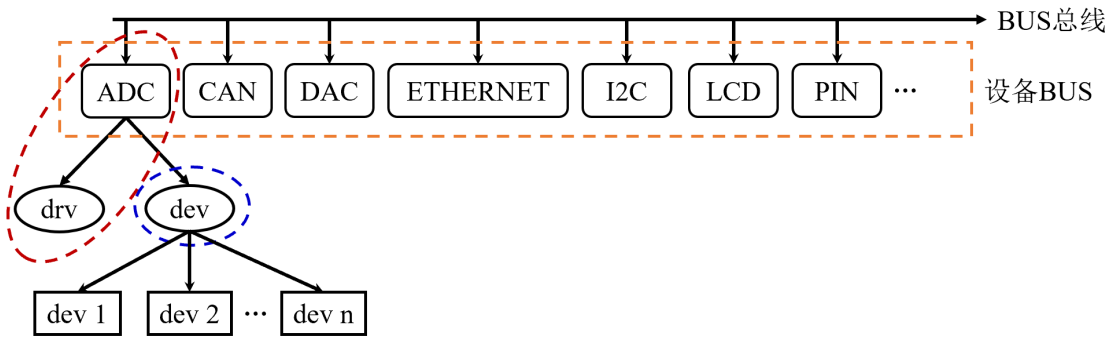


图 3-7-8 驱动框架结构

XiZi 内核的驱动模型如图 3-6-1 所示。在设备驱动设置一条 BUS 总线，其实质就是一个链表，该总线上可挂接多个设备总线，比如 ADC 设备总线、CAN 设备总线、DAC 设备总线等等，即每个设备通过各自的总线与 BUS 总线连接，例如 ADC 设备通过 ADC_BUS 挂接到 BUS 总线上。每个设备总线都关联 drv（驱动）和 dev（设备）结构体。在实际场景中，每种设备驱动都会关联多种物理设备，如图中的 ADC 设备驱动，不同的 ADC 物理设备 dev1、dev2 等，都在 ADC 的 dev 上进行统一管理。

同时，所有设备总线、设备、设备驱动均定义了各自名称，如 ADC1_BUS_NAME 对应的“adc1”、ADC1_DRIVER_NAME 对应的“adc1_drv”、ADC1_DEVICE_NAME 对应的“adc1_dev”，结合 3.5 章节中虚拟文件系统的设计，应用层通过对路径“/dev/adc1_dev”的操作即可实现对内核驱动中 ADC 设备的操作，对应“一切外设皆文件”的设计理念。

3.6.3 驱动模型接口

XiZi 内核中定义了一系列驱动模型函数接口，包含设备总线初始化、设备总线注册、设备总线配置等，如下所示：

表 3-6-1

函数原型	**int BusRegister(struct Bus *bus)**
函数功能	注册设备总线 BUS
参数说明	参数 bus 是需要注册的总线所对应的结构体指针
返回值	返回 EOK

表 3-6-2

函数原型	**int BusUnregister(struct Bus *bus)**
函数功能	删除设备总线 BUS
参数说明	参数 bus 表示是需要删除的总线所对应的结构体指针
返回值	返回 EOK

表 3-6-3

函数原型	**int DriverRegisterToBus(struct Bus *bus, struct Driver *driver)**
函数功能	设备驱动 DRIVER 注册并挂接到设备总线 BUS 上
参数说明	参数 bus 是目标总线所对应的结构体变量指针；参数 driver 是需要注册的驱动

	所对应的结构体指针
返回值	返回 EOK

表 3-6-4

函数原型	**int DriverDeleteFromBus(struct Bus *bus, struct Driver *driver)**
函数功能	将设备驱动 DRIVER 从挂载的设备总线 BUS 中删除
参数说明	参数 bus 参数 bus 是目标总线所对应的结构体变量指针；参数 driver 是需要删除的驱动所对应的结构体指针
返回值	返回 EOK

表 3-6-5

函数原型	**int DeviceRegisterToBus(struct Bus *bus, struct HardwareDev *device)**
函数功能	将设备 DEVICE 注册并挂接到设备总线 BUS 上
参数说明	参数 bus 是目标总线所对应的结构体变量指针；参数 device 是需要注册的设备所对应的结构体指针
返回值	返回 EOK

表 3-6-6

函数原型	**int DeviceDeleteFromBus(struct Bus *bus, struct HardwareDev *device)**
函数功能	将设备 DEVICE 从设备总线 BUS 中删除
参数说明	参数 bus 是目标总线所对应的结构体变量指针；参数 device 是需要删除的设备所对应的结构体指针
返回值	返回 EOK

表 3-6-7

函数原型	**BusType BusFind(const char *bus_name)**
函数功能	在系统中查找设备总线 Bus 结构体
参数说明	参数 bus_name 是需要查找的设备总线名称
返回值	若成功，返回 Bus 数据结构，若失败则返回 NONE

表 3-6-8

函数原型	**DriverType BusFindDriver(struct Bus *bus, const char *driver_name)**
函数功能	在系统中查找设备驱动 Driver 结构体
参数说明	参数 bus 表示从 bus 指向的结构体进行查找；参数 driver_name 是需要查找的设备驱动名称
返回值	若成功，返回 Driver 数据结构，若失败则返回 NONE

表 3-6-9

函数原型	**HardwareDevType BusFindDevice(struct Bus *bus, const char *device_name)**
函数功能	在系统中查找设备 HardwareDev 结构体
参数说明	参数 bus 表示从 bus 指向的结构体进行查找；参数 device_name 是需要查找的设备名称

返回值	若成功，返回 HardwareDev 数据结构，若失败则返回 NONE

表 3-6-10

函数原型	uint32 BusDevOpen(struct HardwareDev *dev)
函数功能	打开设备
参数说明	参数 dev 表示所需要进行打开操作的设备所对应的结构体指针
返回值	若成功，返回 EOK，若失败则返回 ERROR

表 3-6-11

函数原型	uint32 BusDevClose(struct HardwareDev *dev)
函数功能	关闭设备
参数说明	参数 dev 表示所需要进行关闭操作的设备所对应的结构体指针
返回值	若成功，返回 EOK，若失败则返回 ERROR

表 3-6-12

函数原型	uint32 BusDevWriteData(struct HardwareDev *dev, struct BusBlockWriteParam *write_param)
函数功能	往设备写数据
参数说明	参数 dev 表示所需要进行写操作的设备所对应的结构体指针；参数 write_param 表示进行写操作时所传入的参数
返回值	返回 EOK

表 3-6-13

函数原型	uint32 BusDevReadData(struct HardwareDev *dev, struct BusBlockReadParam *read_param)
函数功能	从设备读数据
参数说明	参数 dev 表示所需要进行读操作的设备所对应的结构体指针；参数 read_param 表示进行读操作时所传入的参数
返回值	返回读数据的长度

表 3-6-14

函数原型	uint32 BusDrvConfigure(struct Driver *drv, struct BusConfigureInfo *configure_info)
函数功能	设备驱动配置函数
参数说明	参数 drv 表示所需要进行配置操作的设备所对应的 Driver 结构体指针；参数 configure_info 表示配置信息，包括命令类型和参数
返回值	返回 EOK